

README | ai provider and council | numerical models and growth | game backend and state machines | system data and evidence | gaps and next iteration | reproducibility

Offline reading copy. Scientific figures, rendered state diagrams and available rendered equations are embedded. External source/evidence links require GitHub access.

FarmTact: scientific implementation report

AI orchestration, numerical planning, crop development and evidence boundaries

Audit date: **11 September 2026** · Source baseline: a96025e · Published v7 source: 6efbbfd ·
Study type: **source-code and engineering-evidence audit**, not an agronomic trial.

Reading copies: PDF · offline HTML. The Markdown chapters below are the maintained source.

Abstract

FarmTact combines a visual synthetic-farm application, local statistical baselines, a constrained scheduling solver, deterministic scenario simulation and optional pretrained DeepSeek interpretation. This report traces the implemented pathways rather than inferring capabilities from role names or design requirements. Demand uses an exponentially weighted moving average (EWMA) combined with booked orders. Crop development uses fixed recipe durations and marketable yield assumptions; the farm view derives progress from calendar dates. OR-Tools CP-SAT selects whole-bed planting candidates, and a local inventory simulator evaluates three fixed stress scenarios. Public observations, News and crop literature provide context without automatically becoming numerical forecast inputs.

Acceptance stores a projected outcome and worklist; no farm-time or task-execution engine advances sowing, harvesting or inventory state. The council has distinct planning, conversation and research-study implementations. Default v7 research dialogue is scripted; real numerical jobs and optional adviser calls are separately triggered. Actual LLM calls use a server-only DeepSeek gateway with role/model allowlists, typed output validation, finite reservations and recorded replay. No application-specific ML training, crop physiological model, disease classifier or autonomous farm execution is implemented. Archived tests support specific engineering properties. In particular, both recorded v7 adviser responses remain unsupported; those probes do not establish answer quality. The report provides equations, architecture figures, source traces, reproducibility guidance and a prioritized requirements register for the next immutable application edition.

Contents

1. Scope, method and evidence
2. System overview
3. AI provider and council mechanics
4. Numerical models and plant growth
5. Complete game backend and state machines
6. System, data lineage and evidence

7. Results and interpretation
8. Limitations and next-iteration requirements
9. Reproduction and verification

The linked chapters form one report. Each includes implementation references; the documentation index covers the wider repository.

1. Scope, method and evidence

1.1 Research questions

- Which user actions can cause provider inference, through which roles and models?
- How do the three council workflows obtain context, validate answers and persist results?
- What is calculated locally, and what does “plant growth” mean in the application?
- Which inputs are observed, synthetic, curated, inferred or simulated?
- What do the tests establish, and which requirements remain unimplemented or unvalidated?

1.2 Audit protocol

The audit inspected the committed source, runtime configuration, fixture generator, contracts, numerical solver/simulator, API workers, frontend state mappings, release manifests and dated reports. Three bounded specialist audits covered AI, numerics and system/data architecture; the integrating author reconciled findings into this report, the README and the specifications. These are code-review roles, not application council calls or human study participants.

Read-only GitHub and Fly inspection established repository access, source state, the active Machine and public edition registry. Offline tests and deterministic figure generation are recorded separately in the documentation audit record. No new metered adviser experiment, deployment, farm-state mutation or full browser usability study was performed for this documentation task. Existing reports were read as historical observations, not relabelled as newly reproduced results.

1.3 Evidence hierarchy used in this report

Label	Meaning	What it cannot establish
Source-inspected	Behavior traced to a function, contract, configuration or UI mapping	That every branch was executed successfully
Fresh offline check	Named command run during this audit against local code/fixtures/mocks	Current external account capability or production performance
Archived experiment	Dated committed report from an earlier execution	Present availability, a different source commit or unseen inputs

Label	Meaning	What it cannot establish
External documentation	Provider/tool documentation, with retrieval context	Account-level feature access or successful application integration
Requirement / recommendation	Desired future behavior with an acceptance criterion	Implemented or validated capability

A schema-valid response is not automatically factually supported. A solver's feasible result is not automatically globally optimal or commercially desirable. A repeatable synthetic experiment is not external validation against real crops, orders or human decision-making.

2. System overview

FarmTact: numerical authority and optional AI interpretation

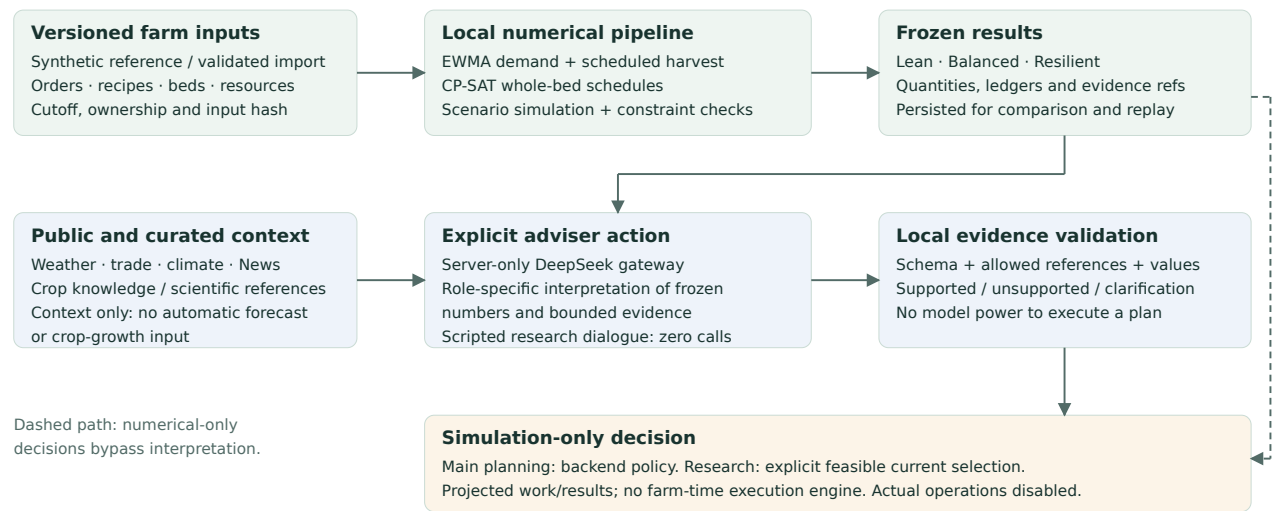


Figure 1. Source audit, 11 September 2026. Solid arrows show implemented information flow; no physiological model is implied.

Figure 1. Source-derived information flow. Numerical tools create the authoritative quantities. Frozen results and bounded context reach optional DeepSeek interpretation. Local checks and the relevant backend decision path control simulation acceptance. Scripted dialogue is a distinct zero-inference interaction. Source: `app.py`, `council.py`, `council_research.py` and `engine.py`. PNG · SVG.

2.1 Mechanism classification

Mechanism	Inputs → outputs	Learned from FarmTact records?	Provider call?
Fixture generation	Fixed parameters → fictional orders/batches/history	No	No
EWMA	Available historical orders + bookings → weekly residual demand	Recursive statistical state; no trained regressor	No

Mechanism	Inputs → outputs	Learned from FarmTact records?	Provider call?
Harvest calendar	Batch dates/recipe assumptions → harvest dates and marketable kg	No	No
Farm artwork	Snapshot planning date + batch dates → stage/progress	No	No
CP-SAT	Candidate schedules/resources → whole-bed allocation choices	No	No
Inventory simulation	Allocations + dated demand + stress cases → ledgers/metrics	No	No
Scripted research dialogue	Bounded intent/context → proposal or clarification	No	No
DeepSeek advisers	Frozen context + role prompt → typed interpretation	Pretrained external model; no app fine-tuning	Yes, explicit paths
Synthetic vision probe	Generated image → checked batch-label observation	No FarmTact training	Yes
News collector	Allowlisted RSS bytes → validated metadata cache	No	No LLM inference

A registered `satellite_visual_reviewer` or `test_evaluator` route grants a bounded capability to trusted server code; it does not establish a deployed satellite pipeline or an LLM evaluation service. See the call-site inventory in the AI chapter.

2.2 Plant development and temporal meaning

Plant development is a recipe calendar

Illustrative common sow date = day 0. Existing batches retain their recorded dates.

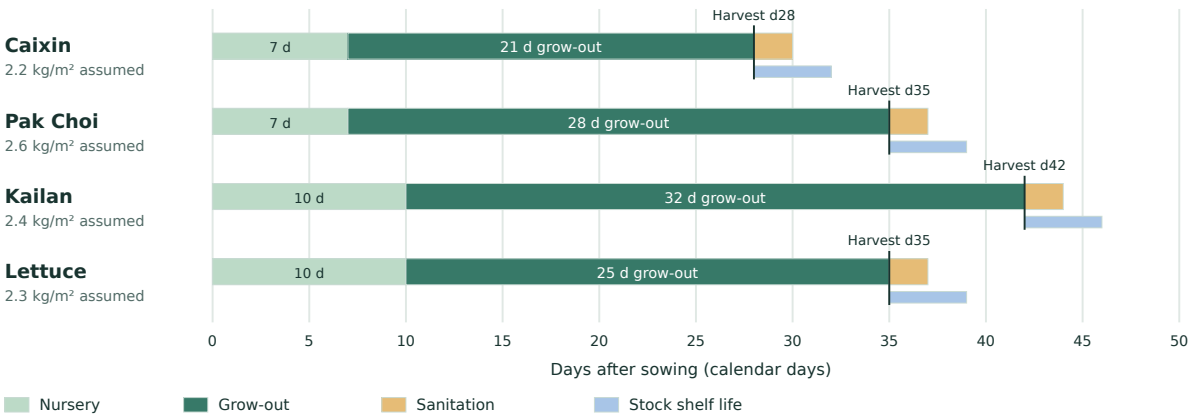


Figure 2. Continuous spans illustrate durations. Daily bed occupancy includes harvest + sanitation through that day. The earliest next transplant is harvest + sanitation + 1 day. Shelf-life expiration is a separate inventory rule.

Figure 2. The four fixture recipes shown on a common illustrative sow date. Nursery and grow-out durations determine harvest. Sanitation affects bed reuse; shelf life affects harvested inventory. Yield coefficients are **synthetic engineering assumptions**. The chart is not a measured biological growth curve. Source: fixtures.py, contracts.py and engine.py. PNG · SVG.

For a batch with sow date s , transplant date t , harvest date h and current snapshot date d , the backend view computes $\text{progress} = \text{clamp}((d - s) / (h - s), 0, 1)$ and chooses nursery/growing/ready from those dates. Artwork adds presentation categories, not biomass measurements. A long calculation does not move d . Research dialogue does not secretly accelerate crop maturity. See the numerical chapter for inclusive-day occupancy, expiry, yield shocks and the limits of simulated execution.

2.3 Demand baseline and interpretation

Synthetic demand: observed fixture values and one-step EWMA

Caixin · alpha = 0.35 · fabricated repeating history · no public/weather features

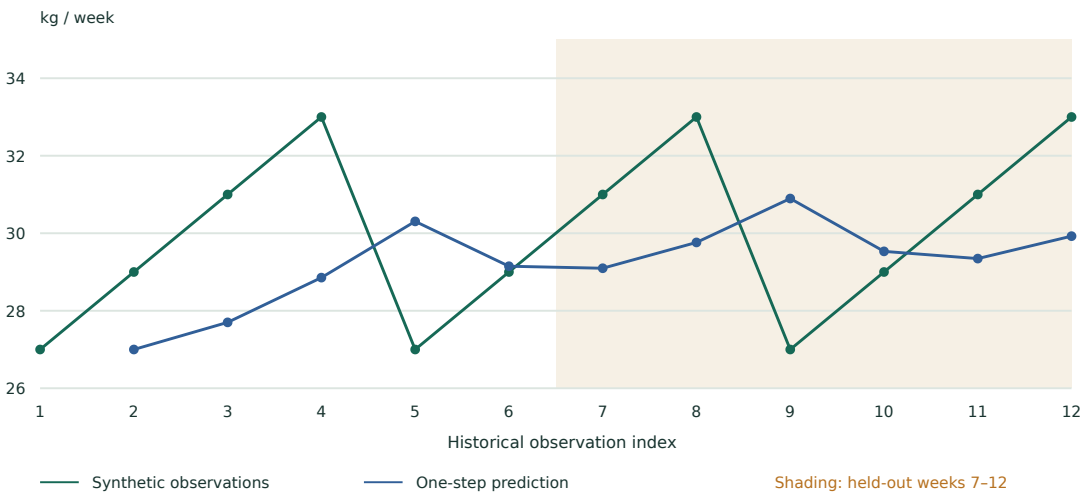


Figure 3. Latest EWMA level = 31.001183 kg/week. Future expected demand = bookings + max(0, level - weekly bookings).

Figure 3. Freshly generated illustration from the source fixture. The blue one-step forecast uses only preceding observations; the shaded observations match the six held-out weeks used by the archived numerical evaluator. The newest EWMA level is used across future weeks, with already booked quantities preserved and only positive residual demand added. A repeating synthetic history can make a baseline appear accurate without establishing performance on real orders. Source: `models, evaluate_numerical.py`. PNG · SVG.

All figures have source SVGs, raster exports, accessible titles/descriptions and textual explanations. Generation code and underlying fixture/forecast data support reproduction. Architecture annotations are authored from source inspection; no generative-image model or inferred experimental data was used to create these figures.

3. Results and interpretation

3.1 Numerical evidence

The archived numerical evaluation compares EWMA with a last-week baseline over six held-out weeks per crop. Caixin EWMA MAE is approximately **2.383 kg/week**, versus **2.667 kg/week** for last-week naive; WAPE is approximately **7.77%** versus **8.70%**. These are synthetic targets from a deterministic four-week pattern. The four crops share the pattern with level offsets, so they do not constitute four independent real-world datasets.

The same archive records constraint-checked strategy outputs and time-limited solver statuses. Its FEASIBLE statuses should not be translated into “the optimal farm plan.” Whole-bed schedules may leave both shortages and surplus because crops, dates and shelf life constrain allocation. The locally calculated minimum across three stress cases is a scenario downside, not a statistical lower quantile. Financial outputs use assumed prices and costs; they are not audited profit.

3.2 Adviser evidence

The v7 local and public probes each made one actual request. Their context/replay/isolation checks passed, but both responses remained **unsupported**. This distinction is central: transport correctness and budget control are necessary engineering properties; they do not validate agronomic interpretation. The older authenticated trial summary retains its own tested roles, models, budget and dates.

The app's capability display also relies on archived trial records rather than a fresh provider probe. The gap register requires the next iteration to distinguish historical verification, current configuration and recent successful execution without generating paid background calls merely for badges.

3.3 Interaction and infrastructure evidence

The v7 report records 111 public browser checks and separate novice-agent checks. Automated completion times, screenshots and AI reviewer opinions do not establish farmer learning, enjoyment, actual screen-reader support or physical touch/keyboard comfort.

The archived shared-capacity experiment completed two concurrent v6/v7 numerical jobs in approximately 22 seconds each and sampled at least 2.34 GiB available memory. This is evidence for that bounded workload, not sustained scalability. The 11 September read-only inspection found one running Fly Machine, 1/1 health checks and seven published editions. No new concurrency experiment was performed.

4. Discussion and next iteration

FarmTact's strongest implemented separation is between numerical authority, contextual interpretation and simulation-only decisions. Its most consequential open issues are adviser evidence quality, truthful capability labels, temporal and metric precision, realistic out-of-sample validation, and human evaluation. Adding more role names or source cards cannot resolve these issues by itself.

The gap register assigns stable IDs, implementation evidence, proposed changes and measurable acceptance criteria. The specifications now reference those IDs and explicitly distinguish a current-state correction from a requested future capability. The existing four-recipe demonstration remains valid as a bounded engineering fixture; new crop recipes, observed growth response, learned demand models and real operations each require their own data and validation.

5. Reproducibility and references

See reproducibility for commands and environment prerequisites, and the audit record for commands actually executed. The detailed chapters provide function-level source links and external references where relevant. Source links target repository-relative files; the baseline commit at the top defines the inspected implementation. Frozen release manifests identify the distinct deployed source/image.

The report does not use archived pass counts as a substitute for new scientific experiments. It supplies the implementation account and acceptance criteria needed to design those experiments responsibly in the next iteration.

AI provider and council implementation report

Audit date: 11 September 2026 (UTC)

Audited revision: a96025e49226d8ca641cf07b49192bd85b464805

Scope: the DeepSeek provider boundary, mission and conversation councils, the playable council study, vision, numerical forecasting and simulation interfaces, evaluation evidence, persistence, replay, budgets, admission control, and egress.

This chapter is part of the FarmTact: scientific implementation report. Consolidated issue IDs, priorities and next-iteration acceptance criteria are maintained in the implementation gap register; this chapter supplies the detailed call-path evidence behind its AI-* entries.

Safety and interpretation boundary. FarmTact is a synthetic planning game in `execution_mode=test`. Its generated text is advisory, its crop parameters are demonstration fixtures, and accepted worklists are for simulation only. Nothing in the audited implementation authorizes a physical farm action.

Abstract

FarmTact is a hybrid decision system. A local statistical baseline forecasts unbooked demand; a local CP-SAT planner constructs three whole-bed policies; a deterministic daily simulator checks timing, inventory, labour, cash, waste and service; and, only after an explicit user request, DeepSeek models interpret a frozen copy of those results. The language models neither calculate strategy totals nor directly mutate the farm. A backend evidence gate checks model output before the development policy can accept a strategy for simulation.

The current council has seven roles. Five use the configured `deepseek-v4-flash` route and two use `deepseek-v4-pro`. A complete main-mission council normally makes seven provider requests and permits at most two additional schema-repair requests. Optional vision adds one request, but that workflow reads an artificial batch label; it does not assess a plant. Persistent conversations make one request for a direct question, two for an invitation exchange, or seven for a full council, before bounded repairs.

There is no learned crop-growth, yield, disease, nutrient, or climate-response model in the current application. “Growth” on the farm board is a linear schedule progress indicator derived from declared sow, transplant and harvest dates. Marketable yield is a fixed recipe coefficient multiplied by bed area, then perturbed only by declared synthetic scenarios. These boundaries are central to interpreting both the interface and evaluation results.

Questions and method

This audit answers five questions:

1. What can trigger paid inference, which model does each route use, and how many HTTP requests can it create?
2. How do council outputs affect a planning decision?
3. Which parts are statistical estimation, mathematical optimization, deterministic simulation, scripted dialogue, or language-model inference?
4. What is persisted, replayed, validated, rate-limited, and allowed to leave the process?
5. Which repository claims are stale, incomplete, or stronger than their evidence?

The method was static source tracing from each public API trigger through job creation, frozen inputs, gateway construction, response validation, persistence, and replay. It also inspected checked-in trial artifacts and compared provider claims with DeepSeek's official documentation. No paid inference request, Fly deployment, or farm operation was made for this audit. Implementation facts take precedence over prose and historical reports. A checked-in PASS file establishes only what that particular artifact records; it is not a current provider health check.

System boundary



This separation is visible in the implementation: missions call the local `plan()` pipeline before considering the credential or reserving an inference budget. The council receives calculated strategies and exact references, and the development policy performs its own eligibility and evidence checks (`services/api/app.py`).

Terms used in this report

Term	Meaning in FarmTact	Provider call?
Forecast	A per-crop EWMA of historical demand, converted to residual demand after confirmed orders	No
Optimization	A bounded CP-SAT search over whole-bed crop allocations	No
Simulation	Daily stock-flow accounting under three declared yield/demand scenarios	No
Schedule preview	Date interpolation and stage artwork in the UI	No

Term	Meaning in FarmTact	Provider call?
Computed perspective	A deterministic text template over scenario metrics	No
Scripted council study	Rules, buttons and predefined dialogue around local calculations	No
Advisor message/council	Structured natural-language interpretation returned by DeepSeek	Yes
Replay	Reading previously stored messages/events	No

The numerical implementation has its own detailed report in `numerical-models-and-growth.md`. Data lineage and the evidence register are covered in `system-data-and-evidence.md`.

Provider contract

Current repository allowlist

The gateway admits one origin, two endpoints, three model identifiers and fourteen named routes. Configuration is accepted only when it exactly matches the compiled policy; unknown or missing fields, a different origin, an extra route, a larger limit, or a model/route mismatch fail closed (`gateway policy`, `GatewayConfig.load`, `config/deepseek_runtime.json`).

Route	Configured model	Capability	Deployed caller in audited app
demand_analyst	deepseek-v4-flash	text	Mission and conversation councils; direct conversations
weather_analyst	deepseek-v4-flash	text	Mission and conversation councils; direct conversations
market_analyst	deepseek-v4-flash	text	Mission and conversation councils; direct conversations
production_analyst	deepseek-v4-pro	text	Mission and conversation councils; direct conversations
supply_chain_analyst	deepseek-v4-flash	text	Mission and conversation councils; direct conversations
profit_analyst	deepseek-v4-flash	text	Mission and conversation councils; direct conversations

Route	Configured model	Capability	Deployed caller in audited app
planning_chair	deepseek-v4-pro	text	Mission and conversation councils; direct conversations
visual_observer	deepseek-v4-flash-vision-exp	vision	Optional synthetic-label observation
evidence_extractor	deepseek-v4-flash	text	No deployed HTTP call site found; gateway/trial surface only
crop_alias_resolver	deepseek-v4-flash	text	No deployed HTTP call site found; gateway/trial surface only
runtime_researcher	deepseek-v4-pro	text	No deployed HTTP call site found; gateway/trial surface only
document_vision	deepseek-v4-flash-vision-exp	vision	No deployed HTTP call site found; gateway/trial surface only
satellite_visual_reviewer	deepseek-v4-flash-vision-exp	vision	No deployed HTTP call site found; gateway/trial surface only
test_evaluator	deepseek-v4-pro	text	No deployed HTTP call site found; local evaluation does not use it

“Configured” is not synonymous with “used.” The route registry is broader than the current application call graph. The generic two-request tool continuation, model-list request and streaming path are implemented in the gateway and exercised by standalone trials, but the deployed mission/conversation workflows call structured chat and vision only (gateway methods).

Comparison with official provider documentation

DeepSeek's official documentation, checked on the audit date, documents the API origin and current model aliases in its Quick Start, the `POST /chat/completions` request and JSON response mode in Chat Completion, model discovery in List Models, inline image content in Vision, and the continuation treatment of `reasoning_content` in Thinking Mode. The provider also documents tool calls as model requests whose functions are executed by the client, not by the model itself (Tool Calls). These documents support the protocol choices and aliases in the repository as of this audit.

They do **not** prove that the FarmTact deployment's account currently has access, credit, or healthy connectivity. The application does not perform a fresh model or chat probe when it renders capability status. See “Evidence freshness” below.

Request construction and privacy

Every inference request is constructed by DeepSeekGateway with:

- an environment-delivered bearer credential; no credential is accepted from an HTTP client;
- `trust_env=False`, so ambient proxy configuration is not inherited;
- redirects disabled and the origin fixed to `https://api.deepseek.com`;
- the model selected from the server-side role table;
- `execution_mode=test` and `data_mode` limited to `synthetic_demo` or `historical_replay` (mode checks);
- a stable provider `user_id` derived by hashing the internal tenant identifier, prefixed with `farmtact_` (pseudonymization).

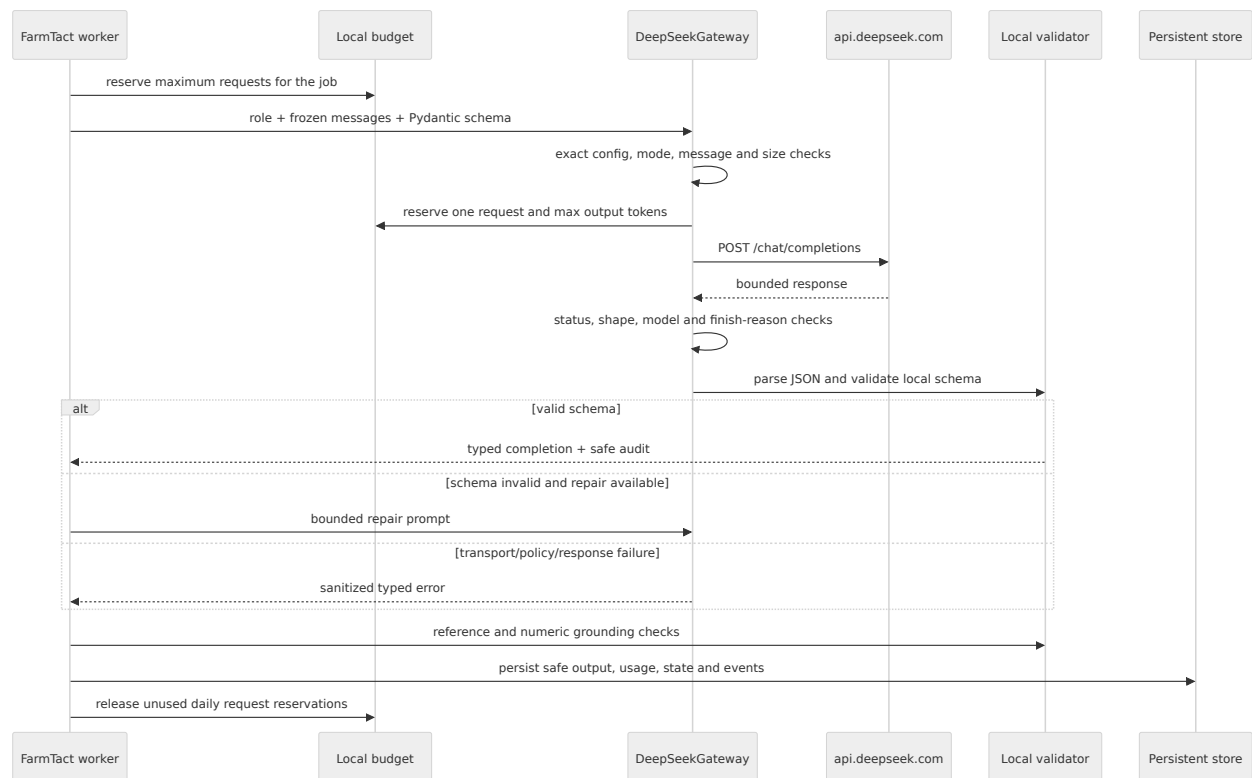
DeepSeek's official Rate Limit & Isolation page permits an alphanumeric, hyphen or underscore `user_id` and says not to send private information in it. FarmTact's hashed pseudonym complies with that shape; the source tenant ID is not submitted as `user_id`. This is pseudonymization, not an anonymity claim: frozen prompts still contain synthetic farm and conversation context needed for the task.

Gateway bounds

Bound	Hard ceiling
Requests in one gateway budget	16
Reserved output tokens	16,384
Concurrent requests	1
Images per request	4
Bytes per source image	4 MiB
Maximum image dimension	4,096 px
Serialized request	16 MiB
Response	4 MiB
Gateway wall time	300 s
Connect/read/write/pool timeouts	10 / 90 / 30 / 5 s

These are code-enforced maxima, not statements about provider-wide limits (limit constants). The budget reserves the declared maximum output tokens immediately before each HTTP request. A failed or shorter request still consumes the local request count and reserved-token allowance; usage is not reconciled downward (`RunBudget.reserve`). That is a conservative token budget, but it is not a currency-cost estimator.

Validation lifecycle



For text JSON, the gateway asks for `response_format={type: json_object}`, accepts exactly one choice with an allowed finish reason, checks the returned model, parses the content, and validates it with the supplied Pydantic type (`chat_json`, response validation). Provider errors are converted to bounded messages and only safe request identifiers are retained (sanitization). Raw request bodies and credentials are not copied into public error records.

The generic tool method permits exactly one allowlisted tool call, validates its arguments locally, runs a local callable, then submits a second request with the tool result. Private `reasoning_content` is retained only in the local continuation and removed from public return values (`run_tool_json`). This method is implemented and trialled, but it is not part of the current deployed council.

Inference triggers and exact request counts

All counts below are provider HTTP requests, not UI turns, agents shown on screen, or local tool executions.

User/API trigger	Normal calls	Maximum calls	Models/routes	Notes
Browse, bootstrap, open advisor, open saved conversation	0	0	—	Read or create frozen state only
Local mission, council=false	0	0	—	Forecast, optimize, simulate and validate locally

User/API trigger	Normal calls	Maximum calls	Models/routes	Notes
Main mission council	7	9	5 Flash + 2 Pro; same role may repeat for shared repair	One call per current role; at most two schema repairs across the council
Main mission council + with_vision=true	8	10	council models + vision-exp	One synthetic label image call before council
Direct conversation message	1	2	selected advisor route	At most one schema repair
Invite advisor	2	3	invited advisor, then original advisor	At most one shared schema repair
Persistent conversation council	7	9	5 Flash + 2 Pro	One sequential turn per role; at most two shared repairs
Direct + invite + council trial sequence	10	14	role-dependent	Aggregate implemented by deepseek_conversation_trial.py; trial budget ceiling remains 16
Playable council-research action or numerical run	0	0	—	Scripted dialogue and local calculation
“Ask DeepSeek” inside completed council research	1	2	selected advisor route	Creates a separate frozen research conversation; one question can require one repair
Planning or conversation replay	0	0	—	Stored events/messages only
Standalone tool-call trial	2	2	configured text route	One model tool selection + one continuation; no deployed call site
Standalone model-list probe	1	1	/models	Diagnostic script only
Standalone stream trial	1	1	configured text route	Diagnostic script only

The main mission reserves exactly 7 + 2, plus one when vision is requested (reservation). Conversation jobs reserve nine for a council and five for either a direct or invite job; unused capacity is returned after the job. Thus direct and invite reservations are more conservative

than their true maxima (conversation worker). The main web client currently submits the council flag but does not expose `with_vision`; synthetic-label vision is an optional API capability rather than part of the ordinary board mission flow (client mission request).

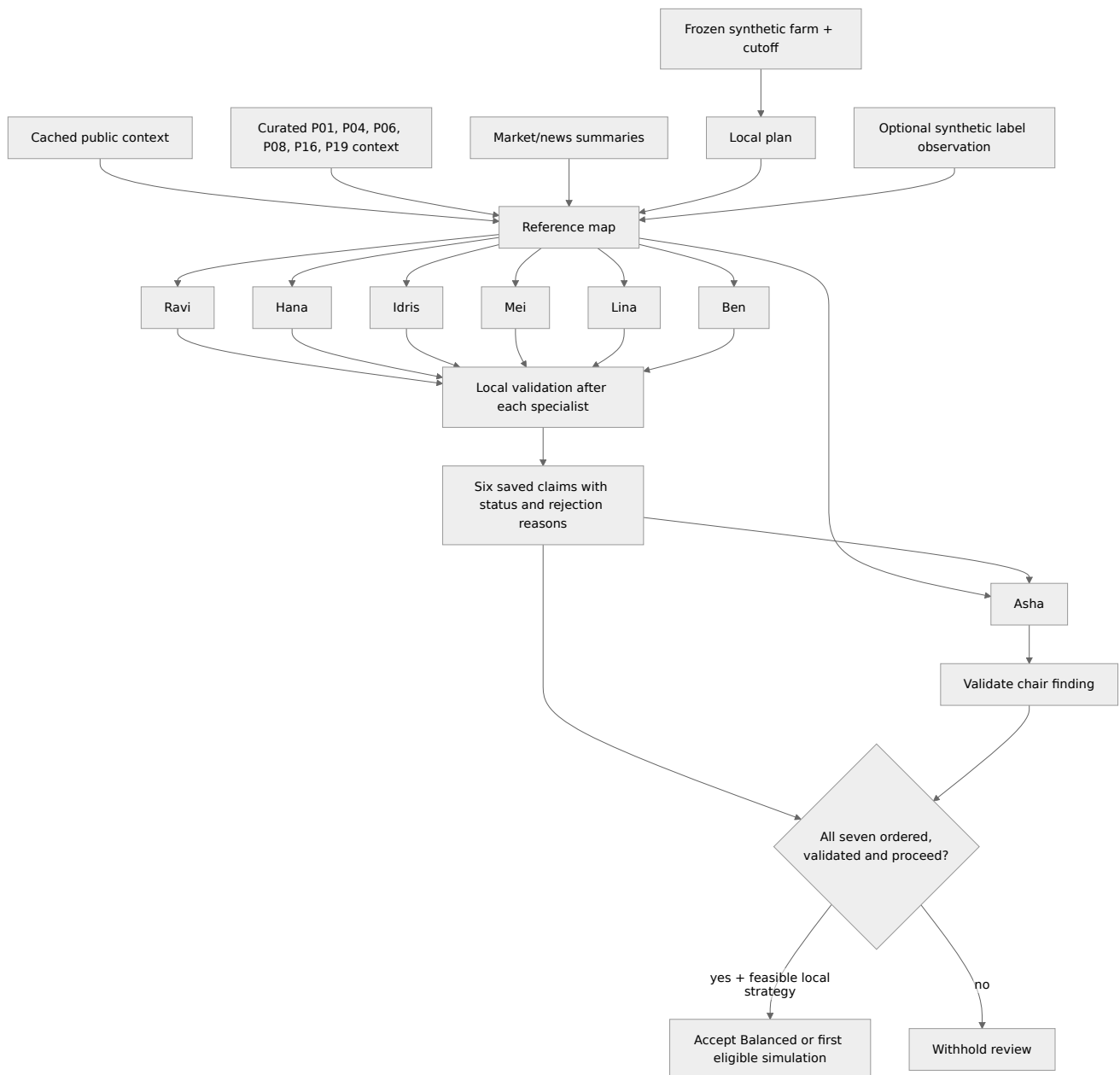
The seven-person council

The authoritative roster and order are defined in `packages/agents.py`:

Order	Person	Role	Model	Frozen responsibility
1	Ravi	Demand analyst	Flash	booked orders, forecast residuals, quantities and dates
2	Hana	Weather analyst	Flash	observed context, freshness and environmental uncertainty
3	Idris	Market analyst	Flash	prices and sourced buyer/grower reactions, without treating reaction as demand
4	Mei	Production analyst	Pro	crop recipes, lead times, space and feasible production
5	Lina	Supply-chain analyst	Flash	inventory, expiry, inputs and delivery timing
6	Ben	Profit analyst	Flash	calculated cost, cash, labour and margin
7	Asha	Planning chair	Pro	reconcile evidence-backed trade-offs among feasible strategies

There is no independent critic in the current roster. The former critic belonged to the archived six-role council. Current automatic checking is ordinary local code, separate from every persona (`council_review_issues`).

Mission council data flow



The council receives each strategy's metrics, constraints, forecast cutoff and uncertainty, the lead-time rule, weather scope, market summary, frozen news evidence, and six selected research-register entries (context assembly). Every role is called sequentially and returns exactly one `Claim` object. The prompt asks for a concise, role-specific finding; thinking is explicitly disabled. Only Asha receives the six preceding claims as `prior_claims`. The six specialists do not see one another's claims (council loop). This is a sequential roster with a final synthesis context, not a multi-round debate or two challenge rounds.

The timing of validation matters. Each specialist claim is schema-checked, then reference-checked and augmented with status and rejection_reasons before it is appended. Because Asha runs last, her `prior_claims` includes both accepted and rejected specialist findings and the reasons for a rejection. The chair can respond to that state, but her response cannot retroactively validate an earlier claim. There is no short-circuit: all seven roles are attempted unless cancellation, budget, transport or response failure stops the job.

The word “tool” in council events can be misleading. The model is not offered a function schema and does not decide to run the forecast, planner, source retrieval or database queries. FarmTact runs those operations first and serializes selected results into the model's user message. The `tool_result_refs` in a claim are keys into that precomputed map. This council therefore implements retrieval-augmented structured interpretation, not agent-directed tool execution.

What local validation proves

For each mission claim the backend checks:

- every `tool_result_ref` is in the supplied reference map;
- every `evidence_id` is in the supplied six-document context; and
- each numeric literal in the statement exactly equals a numeric value in at least one cited scalar tool reference.

It then marks the claim validated or rejected (claim checks). This is useful structural grounding. It does not prove that a qualitative sentence is entailed by the cited record, that a paper is scientifically valid for the current farm, or that the recommendation is agronomically correct. A schema-valid qualitative error can pass unless another deterministic rule catches it.

The two repair allowances apply only when provider output fails JSON/Pydantic schema validation. They do not retry a schema-valid claim with an unknown reference, uncited number or unsupported recommendation. Such a claim is saved as rejected and the council continues. The same distinction holds in persistent conversations: format failure can consume a repair, while reference or action failure produces a saved unsupported message without another provider request.

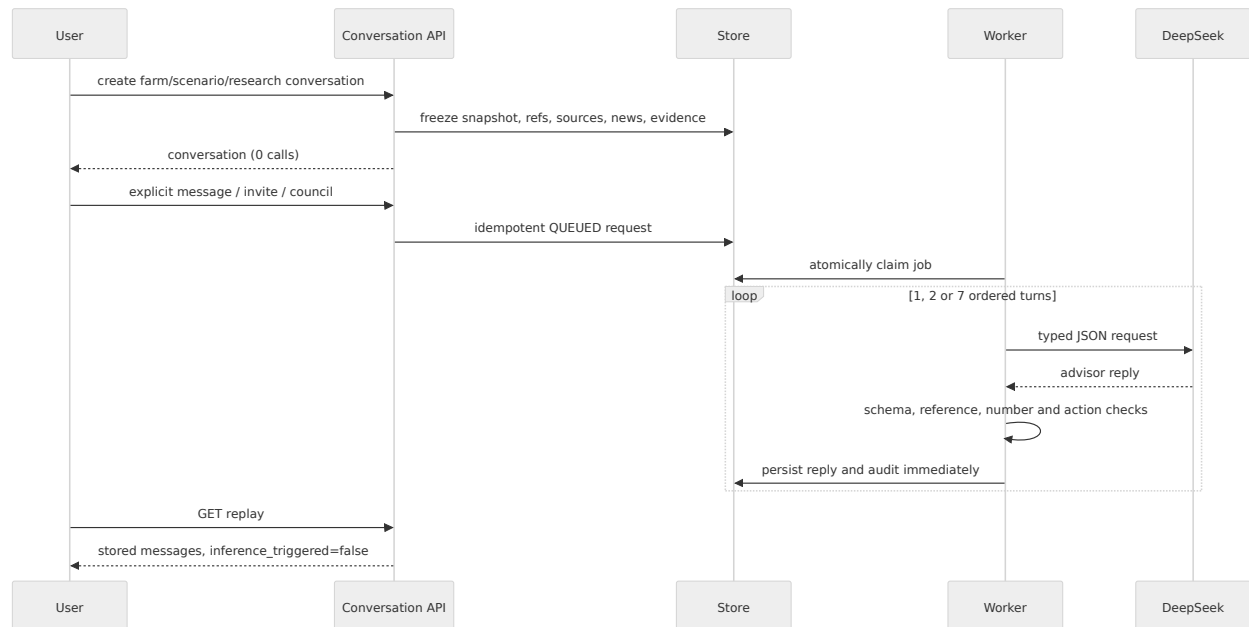
What the council decides

The model does not pick an arbitrary allocation. The backend first keeps strategies that are locally FEASIBLE and pass allocation validation, then chooses Balanced when available. With council claims present, acceptance is withheld unless all seven roles are present in order, all claims passed the checks above, and every recommendation is `proceed_simulation`. With no council claims, the local numerical baseline can be accepted on its own. The result is `ACCEPTED_FOR_SIMULATION`, never an operational commitment (acceptance policy).

This gives natural-language advice a limited but material effect: it can veto an otherwise feasible simulation through a rejected or non-proceeding claim. It cannot make an infeasible strategy eligible, choose an unsupported new strategy, or execute farm work.

Persistent advisor conversations

Persistent conversations are a second inference surface. Creating or reading a conversation freezes and returns state without calling DeepSeek. Explicitly posting a message, inviting an advisor, or convening a council enqueues a job (route definitions).



Frozen context

The snapshot kind is farm, scenario or research. The freezer copies the applicable farm/version/hash, terminal calculation, tool-result map, sources, market signals, news and evidence. A research conversation requires a completed calculation for the current research version; scenario context requires a completed scenario (snapshot freezer). Subsequent model turns operate on these saved values rather than silently reading a newer farm.

The prompt includes at most the last 32 messages and an explicit reply graph, along with frozen tool and evidence values. Retrieved material is labelled untrusted (context construction). An invite first asks the invited advisor and then asks the original advisor to respond. A conversation council calls the seven current roles in order; this council also is not a multi-round debate.

Unlike the mission council, every conversational speaker after the first receives earlier persisted public turns, not only the chair. A reply is locally validated and saved before the next prompt is assembled. Asha therefore sees earlier turns and their `validation_status`, including unsupported, but the context projection does not include the detailed `validation_errors` array. The system prompt says earlier rejected turns are visible; in practice the rejected status is visible while its specific reason must be inferred from the response and references. Each turn's `reply_to` points to the previous advisor reply, forming a chain even though all turns also address the same council question (worker loop).

Conversation models also receive no executable function tools. Their `tool_refs` refer to the frozen lookup table. `proposed_actions` are JSON suggestions validated after generation and displayed as hypotheses; they are not invoked by the worker.

Reply validation and proposed actions

An advisor reply can contain narrative, references and bounded hypothetical action controls. The validator rejects unknown tool, evidence or highlighted-message references. It also rejects numeric literals, spelled-out numbers, dates and relative dates in prose, directing exact quantities into separately rendered reference cards (reply validation). This conversation rule is stricter than the main mission rule, which permits exact cited numeric values.

Action controls are locally bounded to:

- batch delay: 0–14 days;
- batch yield: 50–100%;
- crop demand: 50–150%; and
- labour or cash: 50–150%.

Targets must exist in the frozen snapshot. Actions remain `hypothesis_only` or are blocked; the reply cannot execute them. The prompt requests two short sentences and 400 characters, but the response schema permits 900 characters and the validator does not enforce the sentence or 400-character limits. The prompt also asks for at most three tool refs, one evidence ref, one highlight and one action, while the schema allows 10, 8, 8 and 3 respectively. Reference membership is checked, but those lower prompt cardinalities are not. These are open deterministic-validation gaps (reply schema, `_system_prompt`).

Partial results and restart semantics

Each validated or unsupported advisor message and its audit are persisted after its turn. A later failure can therefore leave a `PARTIAL` conversation with useful saved replies. On process restart, a conversation job that had reached `RUNNING` is marked interrupted and is not automatically requeued, because the process cannot know whether a paid request completed before interruption (restart handling). This is the safe default for uncertain side effects. By comparison, purely numerical scenario and research jobs can be requeued.

`COMPLETED` is the request lifecycle status: it means all requested provider turns were received and persisted. It is not an answer-quality verdict. Each reply carries its own `validation_status` (`references_verified` or `unsupported`) and remains an `unverified_advisor_interpretation` even when references pass. A seven-turn request may therefore be `COMPLETED` while one or more of its saved replies are unsupported (message persistence and request completion).

Replay explicitly reports `execution_mode=replay`, `inference_origin=stored_messages`, and `inference_triggered=false` (public replay).

Playable council research is scripted

The page labelled “Council research” is primarily a deterministic research prototype, not a seven-call LLM council. It uses typed actions and a regular-expression interpreter to reserve a bed, change an order's confirmation state, change a labour limit, challenge an assumption, run a local calculation and choose an eligible simulation. A new session records `inference_calls=0`, `dialogue_mode=scripted_research` and `operational_execution=false` (session creation).

FROZEN-INPUT NUMERICAL EXPERIMENTS

Scenario lab

×

1 Briefing

2 Assumptions

3 Compare

Challenge

Open sandbox



ADVISOR BRIEFING

Open sandbox

Change any supported farm assumption without a quest badge.



RECOMMENDED MISSION · PROVISIONAL SUPPLY GAP

Can we cover Pak Choi due 21 Sept 2026?

28 kg

booked for this date

0 kg

harvest eligible by shelf life

0 kg

inventory valid on due date

28 kg

gap to investigate

This screening comparison uses dated booked orders. It does not allocate an eligible harvest or inventory lot across earlier orders; the numerical planner decides actual shortfall. The containing week's statistical forecast is 33 kg and remains context, not this order's due quantity.

📅

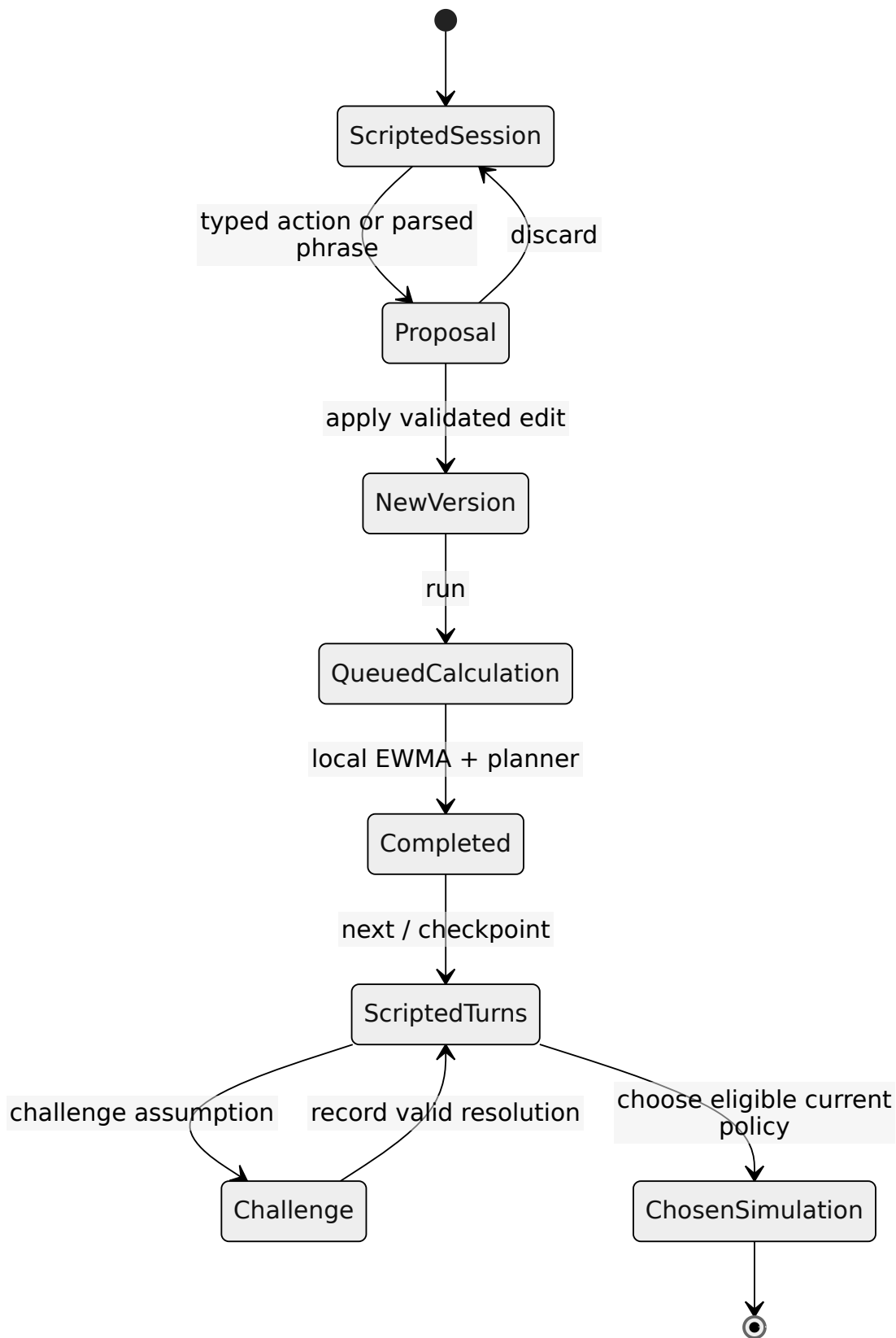
A new sowing using the declared 35-day recipe would mature around 13 Oct 2026—after this delivery.

📖

Inspect 14 linked input records

Snapshot 0028677954

The numerical worker validates a frozen hash and version, calls `calculate_research()` locally, and creates predefined turns only for inputs that changed, followed by a Planner turn. An initial run normally shows Production and Demand plus Planner; changing labour can add Profit. Roles with no changed input do not speak (research calculation). Agreement between those scripted speakers is explicitly not evidence.



Limits are 30 sessions per tenant, 400 actions per session, 12 numerical versions per study, and one queued/running research calculation per tenant workspace (research routes). Choosing a result records a research-participant simulation choice and does not change the main farm.

After a current result completes, the user may deliberately open "Optional actual DeepSeek interpretation." That UI creates a separate persistent conversation with `snapshot_kind=research` and posts one direct question (component). It does not run the full seven-role council. "One

question” normally means one provider request, but one bounded schema repair can make it two.

Forecasting, growth and numerical simulation

What is actually learned or estimated

There is no trained machine-learning model. The only fitted statistical state is a single exponentially weighted moving average per crop:

$$E_t = \alpha D_t + (1 - \alpha)E_{t-1}, \quad \alpha = 0.35 \text{ by default.}$$

Only demand records available before the cutoff and before the planning date enter the recursion. Each future week receives the same EWMA residual after booked demand:

$$R_w = \max(0, E - B_w).$$

That residual is placed on the end date of the week. Confirmed orders remain dated commitments. The forecast labels uncertainty as the fixed synthetic scenarios below; it does not calculate a prediction interval (forecast implementation).

Scenario	Yield multiplier	Residual-demand multiplier	Statistical meaning
Low yield / high demand	0.85	1.15	Declared stress case
Central	1.00	1.00	Declared reference case
High yield / low demand	1.10	0.90	Declared favourable case

The cases each enter the optimizer once and carry an equal declared weight of one third. The solver does not explicitly read the weight field, and the seed argument does not generate samples. They are not learned probabilities, confidence intervals, or calibrated quantiles.

Plant growth is a schedule, not a biological model

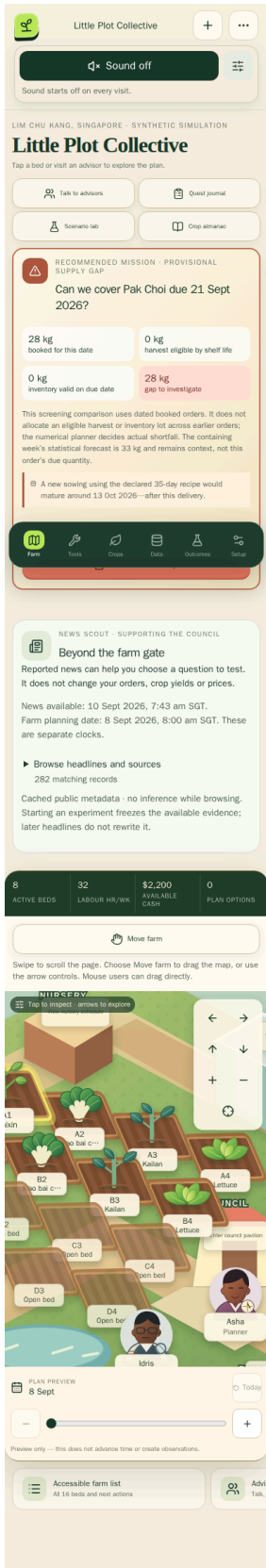
For an existing batch the server derives a stage from recorded dates:

- ready when `harvest_date <= planning_date`;
- nursery before transplant; otherwise
- growing.

Its progress is the clamped linear fraction

$$p = \text{clip}_{[0,1]} \left(\frac{\text{planning day} - \text{sow date}}{\text{harvest date} - \text{sow date}} \right).$$

The farm view implements exactly this calculation (`farm_view`). The frontend preview performs a similar date interpolation and swaps stage artwork. It does not integrate sensor readings, thermal time, light, nutrients, water, humidity, disease, cultivar traits, biomass or observed growth. The UI describes it as schedule preview rather than an observation.



For proposed crops, marketable yield is fixed by the synthetic recipe:

$$Y_{bed} = A_{bed} y_{recipe},$$

where A is bed area and y is declared marketable kilograms per square metre. The simulator applies a scenario multiplier on the scheduled harvest date. There is no growth curve between sow and harvest. Existing-batch forecasts simply copy their declared harvest dates and expected kilograms.

The board and preview also have a boundary inconsistency worth retaining in tests (previewBed): the server labels an existing batch ready on its harvest date, while the frontend's preview renders an existing bed empty at or after its scheduled harvest; new accepted allocations render ready on their harvest date. The two views answer slightly different questions but do not state that difference clearly.

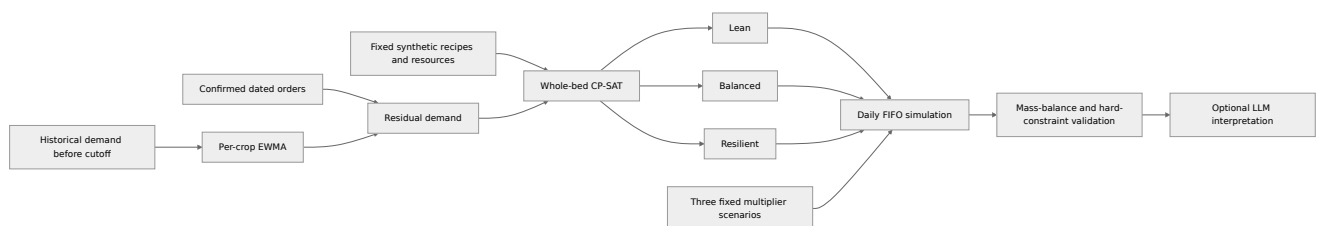
Optimization and daily accounting

The planner builds whole-bed candidates by counting backward from permitted harvest dates through fixed nursery and grow durations. CP-SAT applies occupancy, nursery, weekly labour and conservative cash constraints and solves Lean, Balanced and Resilient policy objectives. It uses one worker, a fixed seed and a four-second limit; a feasible solution need not be proven optimal (planner). A deterministic backward scheduler is used when CP-SAT produces no solution, then passed through the same validation (plan).

The daily simulator:

1. adds scheduled harvests on their declared day;
2. expires lots by shelf life;
3. serves confirmed orders and residual forecast demand from FIFO stock;
4. records unfilled demand without retroactively filling it from a later harvest;
5. calculates fixed input, labour, packing and disposal costs; and
6. checks mass balance and hard constraints.

This is a stock-flow model, not crop growth simulation (daily simulation). Scenario controls can directly move an existing batch harvest date, scale its yield, scale demand and change resource ceilings before a new local plan is run (scenario controls). The crop-demand control scales both historical demand and matching current orders, so it changes the EWMA baseline and commitments together; it should be interpreted as a broad hypothetical branch, not a causal market forecast.



Synthetic disruption and “replan”

The replan API does not ingest a measured plant observation. It takes the first batch in the current synthetic farm, adds seven days to its harvest date and multiplies expected marketable yield by 0.8, saves a new farm version, and replans (replan route). This is a useful deterministic demonstration of versioning and response to a disruption. Calling it observed growth, a detected delay, or a predictive crop model would be inaccurate.

Vision

The only deployed vision path generates an 800×360 PNG in memory containing the words “SYNTHETIC BATCH” and a run-derived FT- label. It sends that image to `visual_observer`, checks that the returned label is exact, and records `agronomic_measurement=false` (`observe_fixture`). The resulting structured description can be included in the text council's frozen references.

The gateway independently validates image count, encoded size, dimensions and supported static formats, rejects animated images and decompression bombs, converts the image to normalized PNG, and transports it inline (image normalization, `vision_json`). No current UI path uploads a crop photograph. The vision feature therefore demonstrates bounded multimodal transport and label extraction only; it gives no evidence of plant health, growth, maturity, disease or yield measurement.

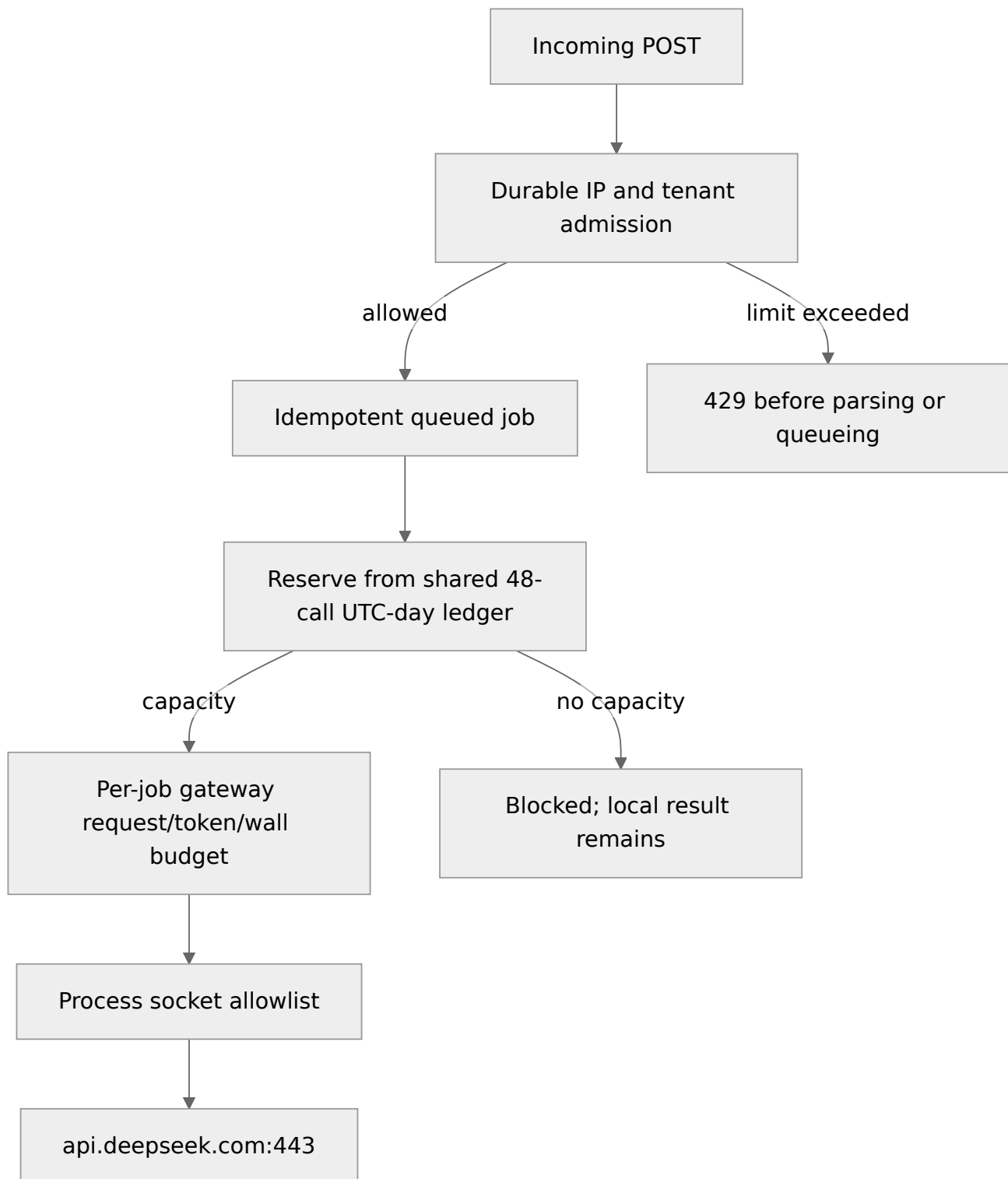
Persistence, provenance and replay

Mission creation stores a frozen farm snapshot and hash, input version, cached source snapshot, research-register version hash, market signals, frozen news, council version, requested mode and parent/disruption metadata before work begins (mission creation). During a mission it persists strategies, claims, safe provider audits, usage summaries and ordered events. The acceptance transaction rechecks the current farm version and content hash; a changed farm invalidates acceptance and starts a fresh local, no-council calculation.

Conversation records likewise preserve their snapshot and input hash, messages, reply graph, references, job status, model, usage and validation status. Replay returns these stored records without new inference. The mission replay endpoint also returns stored events and identifies `inference_origin=stored_events` (mission replay).

The records do not currently carry an explicit prompt-template version on each provider request or output. `council_version` identifies the roster on missions, but it is not sufficient to reconstruct every prompt, schema and validator revision. The generic event `schema_version='1.0'` identifies the event envelope, not the Pydantic advisor/claim schema or prompt revision. Conversation records similarly store model and usage without those semantic versions. Exact reproducibility therefore also depends on the source commit, frozen edition image and checked-in configuration.

Admission, budgets and egress



AI-triggering paths have durable limits of 6 requests per IP per minute, 20 per IP per hour and 12 per tenant per hour. The broader API, write, session, preview and event-stream limits are separate (rules and route matcher). The AI route matcher conservatively includes all planning-run creation and replan posts, including requests whose body later selects `council=false`; these can consume admission allowance even though they cause zero inference.

The persistent daily ledger has a hard ceiling of 48 provider calls. Published editions delegate that ledger to the shared edition-control service; local instances use their own database (daily reservation). Main mission and conversation workers reserve worst-case counts before opening the gateway and release unused requests against the original UTC day afterward.

The service endpoint loads a protected local credential when necessary, installs the process egress policy, then imports the API server (scripts/serve.py). The Python socket policy allows api.deepseek.com:443, the configured PostgreSQL host, validated private edition upstreams when running as a gateway, and a validated .flycast control host (services/api/egress.py). It checks hostname resolution and socket-connect audit events.

This is defense in depth inside the Python process. It is not an operating-system, container, subprocess or infrastructure firewall. Production isolation therefore cannot be inferred from the hook alone. The gateway's fixed HTTPS origin and disabled redirects independently constrain provider HTTP traffic.

Evaluation and checked-in provider evidence

Numerical evaluation

scripts/evaluate_numerical.py runs locally. It performs rolling held-out checks on the synthetic generator, compares EWMA with a last-week naive forecast, and verifies that reported strategies have no recorded violations. It makes no LLM call. Its own limitations correctly say that outcomes and recipes are synthetic, scenario weights are not calibrated intervals, and no real-farm forecast has been promoted. Passing it demonstrates internal consistency on generated data, not agronomic accuracy or external predictive validity.

Provider artifacts and what they establish

Artifact	Recorded time / scope	Recorded requests	What it establishes	Limitation
reports/provider_egress_probe.json	8 Sep 2026 08:29 UTC	1	A model-list request succeeded under the process hook then	Not a current health or chat test
reports/deepseek/latest.json	8 Sep 2026 08:41:30-08:41:44 UTC	15 cumulative	Flash/Pro JSON, generic two-call tool continuation, vision, streaming and an old council trial passed then	Council section is the archived six-role roster, including independent critic
reports/seven_agents/deepseek.json	Seven-agent local candidate; artifact contains no timestamp	10	Direct reply, two invited replies, seven current council replies, stored replay without inference	Cannot be independently dated from the JSON; outputs include unsupported claims

Artifact	Recorded time / scope	Recorded requests	What it establishes	Limitation
reports/deepseek_conversation_live.json	Deployed conversation aggregate; artifact contains no timestamp	11	Saved direct/invite/council messages, grounding states and no-inference replay were observed	Does not by itself identify current account health
reports/v7/advisor_local.json	9 Sep 2026 23:55 UTC event	1	One local v7 advisor boundary run completed	Reply was not reference-verified
reports/v7/advisor_public.json	10 Sep 2026 00:58 UTC event	1	One public v7 advisor run completed and remained isolated	Reply was marked unsupported
reports/fly_integrated_demo.json	Historical integrated script artifact	—	Some component details are inspectable	Overall state is INCOMPLETE; it is not release proof

On a fresh full run, current `scripts/deepseek_trial.py` makes seven base capability requests: model list, Flash JSON, Pro JSON, two-request tool continuation, vision and streaming. `--with-council` adds seven current roster calls, for fourteen normally and a hard ceiling of sixteen. However, the `reports/deepseek/latest.json` file consumed by the capability badge came from the prior six-role version and records fifteen cumulative requests. The script's council scenario is a transport test using toy partner-supply alternatives and a report-local decision event. It is not the deployed planner, which deliberately does not invent partner purchases. A claim that the current seven-role standalone DS-G2 trial has fresh authenticated proof would therefore be unsupported.

Evidence freshness and the capability badge

`capabilities()` labels DeepSeek text verified when two fields in `reports/deepseek/latest.json` are PASS and a credential exists in the current environment. It labels vision verified solely from the archived vision PASS. It does not inspect the artifact timestamp, check the current roster, call `/models`, perform a current account health request, or verify that the credential can use the configured aliases.

Badge field	Actual inputs	Missing from the decision
DeepSeek text status	Archived Flash JSON PASS + archived Pro JSON PASS + current process key presence	Trial age/roster, key validity, account credit/access, current model availability, live request success
DeepSeek model list	Models discovered in the archived report	Current configured-vs-discovered comparison and fresh discovery
DeepSeek overall trial	Archived report's <code>overall_status</code>	Current provider/account state

Badge field	Actual inputs	Missing from the decision
Vision status	Archived vision PASS only	Even current process key presence, plus every freshness/account check above

Consequently the UI badge means approximately “a historical checked-in probe passed, and for text a key is present.” It must not be described as a fresh provider or account verification. A future capability contract should expose at least:

- `historical_trial_status` and `historical_trial_finished_at`;
- `configured_models` separately from models discovered by that trial;
- `credential_present` as a server-side boolean only;
- `current_availability` as unknown unless a deliberately authorized, timestamped health probe has run; and
- roster/trial version so old six-role proof cannot imply current seven-role proof.

Findings and specification corrections

The following items should be treated as corrections or explicit future-work gaps, not silently rewritten as capabilities that already exist.

Priority	Finding	Evidence	Required correction or next iteration
Critical	“Plant growth simulation” currently means fixed date/yield scheduling plus daily inventory flow. There is no biological or learned growth model.	<code>packages/models</code> , <code>farm_view</code> , <code>planner</code>	Rename user-facing/spec claims to schedule and production simulation. Define validation data and equations before adding any empirical growth model.
High	Current council is seven turns plus at most two shared repairs; docs that say eight turns are stale.	<code>packages/agents.py</code> , <code>services/api/council.py</code>	Replace “eight advisor turns plus two repairs” with “seven advisor turns plus at most two shared schema repairs.”
High	The independent critic no longer exists.	Current roster above; historical artifact has six old roles	Remove current-tense “simulated critic” language. Say deterministic backend evidence gate, and label the old critic as archived.
High	Specifications describing two challenge rounds do not match either deployed council.	Mission and conversation loops each call roles once	Specify seven sequential specialist/chair turns. If debate is desired, design and budget explicit rounds rather than implying them.

Priority	Finding	Evidence	Required correction or next iteration
High	A runtime requirement recommends enabled thinking for deep council work, but both current councils explicitly send thinking='disabled'.	services/api/council.py, conversation worker	Either change the requirement to disabled thinking for bounded structured claims, or implement and evaluate an enabled-thinking variant with privacy/cost limits.
High	Latest general capability artifact is an old six-role trial, although current trial code has seven roles.	latest.json timestamps and all_six_roles_present; current script	Separate archived transport evidence from current roster acceptance. Do not claim freshly authenticated seven-role DS-G2 until a deliberately authorized run is recorded.
High	Capability verified status has no freshness or account-health semantics; vision does not even require present credential.	views.capabilities	Publish archived timestamp/status and current availability separately; use unknown when no fresh probe exists.
Medium	Local checks validate schema, reference membership and numeric equality, but not qualitative entailment.	Council and conversation validators	Describe claims as structurally grounded or unsupported. Add deterministic entailment rules or reviewed eval sets before calling them fact-verified.
Medium	Persistent request records lack an explicit prompt/schema/validator version.	Mission and conversation persistence	Store prompt-template version, output-schema version, validator version, route and source commit with every request.
Medium	Conversation prompt says two sentences/400 characters and lower reference/action counts, while the schema allows 900 characters, 10 tool refs, 8 evidence refs, 8 highlights and 3 actions. The validator does not enforce the lower prompt limits.	AdvisorReply, validator	Align the schema and deterministic validator with the documented limits, or remove the stronger prompt claims.

Priority	Finding	Evidence	Required correction or next iteration
Medium	Main council reservations count requests and maximum output tokens, not total monetary cost; cancellation is checked between calls, not an external hard kill.	gateway budget and mission worker	State current semantics exactly. Add price-versioned cost accounting and an independently enforceable job deadline only if required.
Medium	Optional vision reads a synthetic label, not a plant.	services/api/vision.py	Rename it synthetic label verification in capability text. A real crop-imaging feature needs consent, acquisition provenance, task-specific validation and agronomic uncertainty.
Medium	UI/server stage semantics differ around harvest for existing batches.	server farm view and frontend preview	Define whether preview means crop present, scheduled work, or post-harvest bed state; encode one date-boundary contract and test it.
Medium	Demand scenario control scales historical demand and booked orders together.	services/api/scenarios.py	Split forecast-history, unbooked-demand and confirmed-order interventions so causal meaning is visible.
Low	Four configured text helper/evaluator routes and two extra vision routes have no deployed caller; generic tools/streaming are trial-only.	route registry and call-site audit	Mark them reserved/experimental or remove them from product capability claims until an owned workflow exists.
Low	The Python egress hook is described too broadly as the production network boundary.	services/api/egress.py	Call it process-level defense in depth and document infrastructure egress controls separately.

Council assurance and acceptance tests

The council boundary should be reviewed as a chain of independently testable properties. Existing tests cover many structural properties; the remaining quality claims need concrete fixtures rather than provider-success assertions.

Property	Acceptance test	Present evidence / gap
Exact roster and order	Submit a mocked council and assert Ravi, Hana, Idris, Mei, Lina, Ben, Asha exactly once in that order.	Current roster and mission-gate tests cover order; retain as a release gate.
Request ceiling	Force invalid schema responses at different roles and assert no more than two repairs and nine total council calls; add vision and assert ten.	Gateway and conversation repair tests exist; preserve an integrated main-mission maximum test.
Chair context	Return one locally rejected specialist claim and assert Asha's outgoing prompt contains its status and rejection_reasons; assert specialist prompts do not contain earlier claims.	Behavior follows current mission code; add a direct regression because it defines the intended synthesis chain.
Conversation context	Mark an early turn unsupported and assert every later prompt contains its public_validation_status; decide whether to include bounded validation reasons and test the chosen contract.	Status is present, reasons are omitted. Current prompt language is stronger than the projection.
No model tool execution	Capture provider payloads and assert council requests have no tools; verify every cited key already exists in the frozen context.	Architecture currently satisfies this; encode it explicitly to prevent the generic gateway tool path being introduced accidentally.
Numerical authority	Have every model recommend an allocation absent from local results and assert it cannot be accepted or added. Have models approve an infeasible strategy and assert local validation still blocks it.	Backend chooses only locally eligible recorded strategies. Keep adversarial integration coverage.
Veto semantics	Give one otherwise valid claim exclude_unsupported and assert the mission becomes REVIEW_WITHHELD; run the same frozen input without council and assert the local baseline remains inspectable/eligible.	Current council_review_issues has this behavior. A paired test makes the model's exact influence measurable.
Numeric grounding	Generate zero, negative, comma-formatted and decimal literals with and without exact scalar refs. Assert only exact cited values pass; include dates and identifiers so the regex boundary is known.	Mission checks floats by tolerance; conversation blocks all quantitative prose. More adversarial fixtures would document edge cases.
Qualitative grounding	Use a valid reference ID with a statement that contradicts the referenced value and assert rejection.	Not currently implemented. This is the principal grounding gap. Build a typed claim/evidence relation or reviewed entailment fixture before claiming fact verification.

Property	Acceptance test	Present evidence / gap
Research-page truthfulness	Exercise all scripted actions and numerical runs with provider transport set to fail if called; assert zero inference. Then use the explicit DeepSeek action and assert only a separate direct conversation is queued.	Existing council-research tests cover local behavior; retain a transport tripwire in browser/API integration.
Replay	Record completed and partial jobs, replay them with provider transport set to fail if called, and assert byte-stable messages/events plus <code>inference_triggered=false</code> .	Existing replay trials and tests support the invariant. Add it to every edition's release gate.
Restart uncertainty	Interrupt a paid conversation after one saved turn and assert INTERRUPTED/partial preservation with no automatic resubmit; interrupt local research calculation and assert safe requeue.	Current stores intentionally differ; retain database-backed concurrency tests.
Prompt reproducibility	Persist and assert prompt, schema, validator, council and source versions on every generated record.	Gap: only some surrounding versions are stored.
Capability freshness	Feed an expired six-role report plus a credential and assert UI says historical/unknown, not currently verified; feed a timestamped authorized current probe and assert its exact scope.	Gap: current capabilities() reports verified from stale archive fields.
Scientific validity	Evaluate forecasts and any future growth model on held-out real, provenance-controlled farm data with predeclared metrics and uncertainty calibration.	Gap by design: current evaluation is synthetic and cannot establish real-farm accuracy.

Reproducibility checklist

A no-inference review can be reproduced from a clean checkout with these classes of checks:

```
# Static call graph and route inventory
rg -n "DeepSeekGateway|chat_json|vision_json|run_tool_json|list_models|stream_chat" \
  services runtime scripts

# Verify the current roster and exact compiled/configured gateway policy
python -m pytest -q tests/seven_agents tests/deepseek

# Exercise local forecasting, optimization and scenario behavior only
python -m pytest -q tests/numerical tests/council_research
```

Tests should run with provider transport mocked or the provider credential absent. Authenticated scripts under `scripts/deepseek_*trial.py` and the egress probe make real provider requests and require deliberate execution within the authorized budget; they are not part of this documentation audit and are unnecessary to inspect the code paths or replay stored evidence.

Limitations of this report

This is a source and artifact audit at one revision. It does not measure model answer quality, latency, current price, current account access, Fly network policy, physical farm outcomes, or real crop growth. Checked-in screenshots demonstrate presentation at their named viewport and release context; they are not evidence of provider activity or scientific validity. DeepSeek may change its service documentation after the audit date. Repository configuration remains the application authority until a reviewed change is made.

The most defensible description of the present application is: **a synthetic, versioned farm production-planning simulator with a bounded optional seven-role DeepSeek interpretation layer and deterministic safety gates.**

Numerical models, planning, and crop-growth representation

Implementation audit date: 11 September 2026

Current numerical versions: synthetic-farm-v1, recipe-ewma-v1, daily-bed-cpsat-v1, council-research-v1

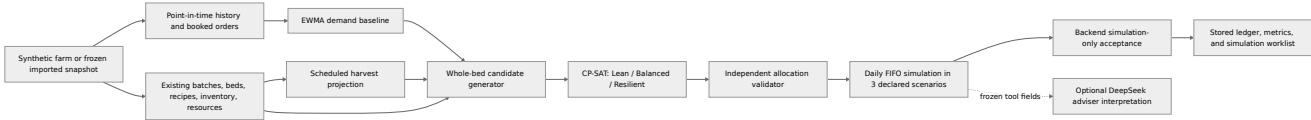
This report describes the numerical behavior in the current repository. It separates implemented calculations from requirements and future research. FarmTact currently combines a deterministic fictional farm, a constant-level exponentially weighted moving average (EWMA), fixed recipe dates and yields, whole-bed constraint optimization, and a daily inventory ledger. It has **no trained or fine-tuned FarmTact model**, no learned plant-growth model, and no validation on commercial farm outcomes. The four crop recipes and all scenario coefficients are engineering fixtures with `validation_status=demo_only` and `origin=synthetic`.

The principal implementation sources are the private-data contracts, fixture generator, forecast baseline, planner and simulator, research wrapper, scenario service, farm view, and world preview. The higher-level inventory in `data-and-models.md` is consistent with this core boundary, but some specification phrases are broader than the implementation; those differences are listed below.

1. What each component actually does

Component	Implemented method	Output	Scientific status
Fixture generator	Closed-form deterministic records	Fictional recipes, beds, batches, orders, history, resources, inventory	Engineering test data; no measured farm observations
Demand forecast	Simple level EWMA per crop	Weekly confirmed and residual kilograms	Statistical baseline evaluated only on synthetic patterned history
Harvest forecast	Copies scheduled batch date and declared marketable mass	Batch/date/kg records	Schedule projection; no fitted maturity or yield model
Growth display	Date thresholds and linear interpolation from sow to harvest	nursery, growing, ready, and percent progress	Calendar visualization; no physiological state estimate
Planner	OR-Tools CP-SAT over Boolean whole-bed candidates	Lean, Balanced, and Resilient schedules	Deterministic optimization over declared assumptions; no ML

Component	Implemented method	Output	Scientific status
Outcome simulator	Daily crop-specific lot ledger, with expiry and FIFO delivery	Harvest, delivery, shortfall, waste, stock, margin, labour	Synthetic counterfactual accounting; no actual outcome prediction validation
Research wrapper	Deep-copies a farm, applies bounded controls, reruns the planner	Versioned, isolated policy comparison	Local numerical experiment; no inference and no main-farm mutation
Runtime advisers	Receive frozen numerical results through the separate DeepSeek gateway	Bounded interpretations and claims	Pretrained model output; advisers do not calculate or validate the plan



Public NEA, SingStat, NASA POWER, and News records are displayed as context but do not enter the current EWMA, harvest projection, scenario factors, or solver coefficients. The feature manifest explicitly records `public_features_used=[]`; see the feature builder and feature manifest.

2. Units, calendar, and numerical conventions

The farm cutoff is a timezone-aware instant. Planning dates are Singapore civil dates obtained by converting the cutoff to Asia/Singapore. The reference cutoff is 2026-09-08T00:00:00Z, which is 8 September 2026 in Singapore. The default horizon is 56 calendar days, represented internally by day indices 0 through 55 and eight week bins.

The contracts use Decimal for physical and financial inputs. Several planner calculations convert values to Python floating point before rounding. CP-SAT coefficients use:

- whole beds and Boolean selection variables;
- grams for mass inside the optimization;
- cents for prices and costs;
- minutes for weekly labour capacity; and
- daily occupancy intervals.

The solver uses one search worker and the farm's `fixture_seed`, which improves repeatability. Its default limit is four wall-clock seconds per policy, so the incumbent can still vary with OR-Tools version, machine speed, or load when search ends before optimality. A returned FEASIBLE result is not claimed to be optimal; the output retains the objective, best bound, elapsed time, and solver status.

3. Reference fixture generation

The fixture generator constructs Little Plot Collective, a fictional sheltered-hydroponic farm in Lim Chu Kang. It contains 16 beds of 20 m², so the declared effective growing area is 320 m². It creates eight active batches, 32 future order lines, 48 historical weekly demand records, and one opening inventory lot. The same data is materialized in `synthetic_farm.json`.

3.1 Synthetic recipes

All four recipes assume 16 plants/m², 0.08 sow-labour hours/m², 0.07 harvest-labour hours/kg, four days of shelf life, and two sanitation days after harvest.

Crop	Nursery (days)	Grow-out (days)	Total lead time (days)	Marketable yield (kg/m ² /cycle)	Input cost (SGD/m ²)	Full 20 m ² bed (kg)
Caixin	7	21	28	2.2	2.00	44
Pak choi	7	28	35	2.6	2.50	52
Kailan	10	32	42	2.4	2.80	48
Lettuce	10	25	35	2.3	3.00	46

These coefficients were declared to exercise scheduling logic. They were not estimated from the papers in the evidence registry and are not claimed to be locally optimal crop recipes.

3.2 Existing batches and inventory

For zero-based batch index i , the scheduled harvest date is

$$h_i = d_0 + 14 \left\lfloor \frac{i}{4} \right\rfloor + 6 \text{ days},$$

where d_0 is the planning date. The first four batches therefore harvest on day 6, and the next four on day 20. Sow and transplant dates are obtained by subtracting the recipe's total and grow-out durations. Every batch occupies a whole 20 m² bed and its expected mass is

$$Y_i = 20 \text{ m}^2 \times y_{r(i)}.$$

The opening stock is 5 kg of caixin, harvested one day before the planning date and explicitly expiring two days after it. The lot ledger uses the supplied expiry date; it does not replace it with a recipe-derived date.

3.3 Orders and historical demand

For crop index $c \in \{0, 1, 2, 3\}$ and future week $w \in \{0, \dots, 7\}$, the unmodified order fixture is

$$Q_{c,w}^{order} = 22 + 4(c \bmod 2) + 2(w \bmod 3) \text{ kg},$$

with price

$$P_c = 6 + c \quad \text{SGD/kg.}$$

Every order is booked seven days before the cutoff and due on day $7w + 6$. There is one order per crop per week in the default fixture.

For historical week $u \in \{0, \dots, 11\}$, the Data Explorer generator uses

$$Q_{c,u}^{hist} = [27 + 2c + 2(u \bmod 4)a]m_h \left(1 + \frac{gu}{11}\right),$$

where a is pattern_amplitude, m_h is history_multiplier, and g is history_trend. The multiplication in code is equivalently base $\times$ multiplier $\times$ trend; the bracket above includes the amplitude-adjusted four-week pattern. With default settings $(a, m_h, g) = (1, 1, 0)$, the four-week pattern repeats exactly. This periodicity is designed, not discovered seasonality.

The bounded playground controls are:

Setting	Minimum	Default	Maximum	Affected records
Historical multiplier	0.50	1.00	1.50	Historical kg
Historical trend	-0.30	0.00	0.30	Historical kg
Pattern amplitude	0.00	1.00	2.00	Historical kg
Order multiplier	0.50	1.00	1.50	Order quantity kg
Price multiplier	0.50	1.00	1.50	SGD/kg

The default resources are 2,560 simultaneous nursery sites, 32 labour hours per week, and SGD 2,200 for the planning horizon. These are declared capacities, not measured availability.

4. Demand and harvest baselines

4.1 Point-in-time filtering

For each crop, history is eligible only when both

$$available_at \leq cutoff \quad \text{and} \quad week < planning_date.$$

Orders are eligible when booked_at $\leq$ cutoff. The Farm contract already rejects future-booked orders, and the forecast repeats this cutoff check. Net booked demand is quantity_kg - cancelled_kg.

4.2 EWMA formula

Let $q_{c,1}, \dots, q_{c,n}$ be the eligible historical weekly kilograms for crop c . The implementation initializes the level with the first value and folds subsequent observations:

$$e_{c,1} = q_{c,1}, \quad e_{c,t} = \alpha q_{c,t} + (1 - \alpha)e_{c,t-1}.$$

The same terminal level $e_{c,n}$ is used for every future week. There is no seasonal index, trend extrapolation, public covariate, buyer model, quantile model, or fitted uncertainty distribution. ForecastSettings accepts finite floating-point $\alpha \in [0.05, 0.95]$; the default is 0.35.

For crop c and planning week w , let $B_{c,w}$ be all net booked kilograms in that week. Residual forecast demand is

$$R_{c,w} = \max(0, e_{c,n} - B_{c,w})$$

when history exists, and zero otherwise. Confirmed lines retain their actual due dates. The entire residual $R_{c,w}$ is added once, on the final day of the planning week. Thus confirmed bookings are not double-counted, but the weekly residual is given an artificial week-end due date.

When several order lines share a due date, their forecast price is the quantity-weighted mean price. A week-end residual without a same-date booking uses the first order price found elsewhere in that week. A week with residual demand and no booked order line receives a price of zero. This is a defined software behavior, not an economic estimate.

4.3 Harvest projection

The harvest baseline simply emits each active batch's recorded harvest_date and expected Marketable_kg, with endpoint fresh Marketable_kg. No inference is made from plant observations. Future planned harvest is similarly

$$Y_j = A_j y_{r(j)},$$

where A_j is the entire bed area and y_r is the fixed recipe yield. The marketable endpoint is assumed to include survival and packout once; there is no second reduction.

4.4 Versioning and lineage

The forecast stores:

- a hash of the complete farm input;
- a configuration hash of model version plus EWMA settings;
- a numerical-input hash combining those two hashes; and
- validation_status=demo_only plus an explicit uncalibrated-uncertainty statement.

The feature builder emits 32 crop/week rows for the reference fixture and records order/history dependency IDs, cutoff, input hash, and version. Saved Data Explorer snapshots additionally freeze the generated farm, generator and forecast settings, forecast payload, reference snapshot, and corresponding hashes. The explorer service rejects a saved snapshot if its schema version, farm contract, settings, dataset hash, frozen forecast hash, or reference hash no longer agrees.

5. Crop-growth representation

FarmTact currently represents a crop cycle as scheduled milestones:



The backend and frontend do not implement the entire diagram as a shared state machine. The planner accounts for sanitation in bed occupancy. The visual farm has no sanitation stage.

The backend farm view computes an existing batch's progress as

$$p(d) = \text{clip}_{[0,1]} \left(\frac{d - \text{sow}}{\text{harvest} - \text{sow}} \right),$$

then labels it nursery before transplant, growing from transplant to the day before harvest, and ready on or after harvest. The interactive preview applies the same linear ratio as a percentage to accepted allocations. It displays nursery, growing, or ready while the preview date is within the allocation's sow-to-harvest interval. For an existing bed without an accepted allocation overlay, it displays the bed as empty at or after its scheduled harvest.

Consequently, “progress” means elapsed calendar fraction between declared dates. It is not measured development, thermal time, accumulated light, biomass, leaf area, stress, quality, disease, or probability of maturity. The crop SVG changes are representative artwork tied to date labels. The interface itself correctly states that the view is a schedule preview and does not claim observed growth.

The scenario system can alter one existing batch by adding 0–14 days to its harvest date and multiplying its declared marketable mass by 50–100%. These are direct counterfactual controls. The main replan endpoint applies a fixed synthetic disruption of +7 days and $\times 0.8$ marketable mass to the first batch. Neither operation derives the delay or loss from weather, sensor data, an image, or a biological model.

6. Candidate schedules and constraints

6.1 Candidate construction

Each candidate j is one full-bed planting for a compatible recipe and harvest date. Candidate harvest dates are every week-end day in the horizon plus any exact booked order due dates. The planner schedules backward:

$$\text{sow}_j = \text{harvest}_j - (\text{nursery}_r + \text{grow}_r),$$

$$\text{transplant}_j = \text{sow}_j + \text{nursery}_r.$$

A candidate is discarded if sowing would precede the planning date, if transplanting would occur before that bed is released by an existing batch and its sanitation period, or if its grow-out/sanitation occupancy intersects a research reservation. New sowing can therefore never be offered as supply for a due date earlier than its recipe lead time.

The decision variable is Boolean:

$$x_j \in \{0, 1\}.$$

There is no fractional bed, partial area, or arbitrary planting-site decision in this version.

6.2 Capacity constraints

For day d , candidate nursery usage is the ceiling of bed area times declared density throughout [sow, transplant). With fixed existing usage N_d^0 ,

$$N_d^0 + \sum_j [A_j \rho_j] I(j \text{ in nursery on } d) x_j \leq N^{max}.$$

Grow-out bed occupancy is inclusive from transplant through harvest + sanitation_days. For each bed b and day d , existing and selected occupancy may not exceed one batch.

Sow labour is charged on the sow-date week at $A_j l_j^{sow}$. Harvest labour is reserved on the harvest-date week using the maximum declared yield factor 1.1 and rounded upward to the next minute:

$$L_j^{harvest} = \lceil 1.1 Y_j l_j^{harvest} \times 60 \rceil / 60.$$

Weekly existing plus candidate labour cannot exceed the farm's declared hours/week.

The CP-SAT cash bound is a conservative planning allowance composed of input cost, labour valued at SGD 12/hour, and a combined SGD 0.45/kg packing/disposal reserve at $1.1 \times$ yield. It is then limited to a policy-specific fraction of declared cash. Existing sunk input cost is not recharged. The post-solve validator also checks the simulated total cost against the full farm cash amount.

The independent allocation validator checks duplicate actions, immutable executed work, dependencies, crop/system compatibility, exact whole-bed area, minimum biological lead time, past sowing, declared yield, reservations, nursery capacity, weekly labour, overlapping bed occupancy, and input cash. Its violations are retained as structured hard-constraint records.

The current data contract does not supply seed availability, water, energy, HVAC, packing capacity, grade, packaging, rotation rules, multiple cuts, partner supply, or crop-specific harvest windows. Although the build specification requires these for a fuller operational planner, the current solver cannot enforce absent fields.

7. CP-SAT objective and strategy families

All policies use the same candidate set and three static scenarios. The policies change four hand-set coefficients:

Policy	Shortage reward/penalty coefficient	Expiry/waste coefficient	New-commitment markup	Cash fraction
Lean	1	9	0.20	0.65
Balanced	4	5	0.04	0.85
Resilient	25	1	0.00	1.00

For each scenario, crop, and day, the model creates age-bucket inventory, delivery, and remaining-stock variables. Supply consists of eligible opening inventory, scenario-scaled existing harvest, and scenario-scaled selected harvest. Delivery is bounded above by due demand. Delivering a gram earns a coefficient based on selling price, shortage preference, and SGD 0.30/kg packing cost. Stock that reaches its final shelf-life age before the last horizon day incurs the policy waste coefficient plus SGD 0.15/kg disposal cost. Selected candidates incur input and reserved-labour cost, multiplied by the policy's commitment markup.

This is best described as a **policy-weighted utility objective**, not a literal expected contribution-margin maximization. The shortage, waste, and commitment weights are engineering preferences. Confirmed delivery is encouraged but is not a hard service requirement. The three scenario weights are stored as 1/3, but the solver loops over the scenarios equally and does not read the weight field. With the current equal weights, this is numerically equivalent to an unnormalized equal-weight expectation; changing stored weights alone would not change the optimization.

The Resilient policy does not solve a maximin or chance-constrained problem. It applies a much larger shortage coefficient across all three scenarios. “Downside” fill rate and margin are calculated after optimization as the minimum result across the three declared cases.

If CP-SAT returns neither FEASIBLE nor OPTIMAL, the planner invokes backward-scheduling-v1. That fallback orders demand rows by week, descending confirmed kilograms, and crop ID; it greedily adds matching-date candidates that pass the independent validator and remain under simulated cash in the high-yield/low-demand scenario. The fallback primarily covers confirmed demand and is explicitly recorded in solver metadata.

8. Scenario simulation and accounting

The fixed scenario set is:

Scenario	Yield factor	Residual-demand factor	Stored weight
Low yield / high demand	0.85	1.15	1/3
Central	1.00	1.00	1/3
High yield / low demand	1.10	0.90	1/3

Yield factors apply to both existing and newly planned harvest. Demand factors apply only to EWMA residual demand:

$$D_{c,d,s} = B_{c,d} + f_s^D R_{c,d}.$$

Confirmed commitments never shrink in the low-demand scenario. The farm seed is stored as `scenario_seed` and is also used as the CP-SAT random seed. The `scenarios(seed)` function does not use its argument to generate a draw, and `scenario_set_id` hashes the same static three-case list rather than the seed.

The outcome simulator evaluates a frozen daily accounting ledger; it does not advance farm time or execute events. For each dated ledger row, expired lots are disposed first, scheduled harvest is then added, due demand is served crop by crop from the earliest-expiring eligible lot, and remaining stock carries to the next calculated row. Harvest-day product can serve harvest-day demand. A newly harvested lot with shelf life k is eligible on the harvest day and the next $k - 1$ civil dates.

For each day, the simulator asserts

$$opening + harvest - delivery - disposal - closing = 0$$

within 10^{-6} kg. It reports weekly demand, harvest, delivery, and same-date shortfall. Later supply cannot repair an earlier day's shortfall.

Central-scenario financial metrics use:

$$revenue = \sum delivered_{c,d} \times price_{c,d},$$

$$cost = input + 12 \times labour + 0.30 \times delivered + 0.15 \times disposed,$$

$$margin = revenue - cost.$$

margin_sgd is therefore a synthetic contribution-like metric under a partial cost fixture. It excludes fixed overhead, tax, depreciation, financing, customer-specific fulfilment cost, and any unmodelled resource cost. area_m2 is the sum of the areas of all new allocations, so repeated use of one bed in separate cycles is counted repeatedly; it is planted cycle-area, not unique land footprint or peak occupied area.

The accepted simulated_outcome is the selected strategy's **central** ledger, metrics, and proposed future work. Acceptance freezes a projection and bookkeeping record. It does not run a farm-time clock, execute the dated ledger as events, mutate batches as work occurs, or create observed biological or commercial outcomes.

9. Isolated scenarios and council-research wrapper

9.1 Scenario Lab

The scenario service deep-copies a frozen farm and supports:

- harvest delay: 0-14 whole days;
- expected batch yield: 50-100% of the frozen value;
- crop demand: 50-150%, applied to both order quantities/cancellations and historical demand for that crop;
- weekly labour: 50-150%; and
- planning cash: 50-150%.

Each branch stores its input snapshot, hash, baseline root, numerical versions, output, same-policy deltas, and zero inference calls. A changed crop-demand control scales both history and orders, so it changes both booked kilograms and the EWMA level. Completed branches are

comparable only when their frozen baseline root, baseline hash, baseline forecast settings, and planner version agree. A branch result never replaces the main farm or its accepted worklist.

The backend selects a feasible Balanced branch for simulation when available, otherwise the first feasible policy. This branch acceptance is local and simulation-only.

9.2 V7 council research

The research wrapper accepts inclusive in-horizon bed reservations, a unique list of unconfirmed order IDs, and a whole-number labour ceiling from 50–100% of declared labour. It rejects reservations that intersect an executed batch from transplant through harvest plus sanitation.

Calculation occurs on a deep copy:

1. listed unconfirmed orders are removed from the booked-order set;
2. labour capacity is multiplied by the selected percentage;
3. the ordinary EWMA and three-policy planner run with the reservation windows; and
4. the result is bound to the base-farm hash, numerical-input hash, controls, and council-research-v1 version.

Removing an order does not erase historical demand. Its quantity can reappear partly or fully as EWMA residual demand, which is displayed separately from booked commitments.

The research service freezes each job's farm, input version, controls, model versions, and hash. It permits at most 12 numerical versions per study and only one queued/running research calculation per workspace. A late result for an older version is archived and cannot become current. If a later version runs before the unchanged version-1 reference was calculated, the worker calculates and stores that baseline too. Scripted research dialogue and calculations make no model request; an explicit adviser conversation is a separate action.

10. Evidence and evaluation

10.1 Archived repository evidence

The checked-in numerical evaluation is an archived run for the original fixture hash 002867...f76. It reports six held-out synthetic weeks per crop. EWMA MAE is about 2.383 kg for every crop, compared with 2.667 kg for a last-week naive baseline. Crop WAPE ranges from about 6.50% to 7.77% for EWMA. This result only shows that the implementation can replay a predictable synthetic four-week pattern. It is not evidence of demand accuracy on an SME farm.

The same archived report records feasible, non-optimal four-second solver results for all three policies and daily mass-balanced central ledgers. Those strategy values belong to that exact fixture, software state, time limit, and OR-Tools run. They are examples, not stable benchmarks or farm-performance claims. Other archived release reports record wider API/browser/persistence checks; see `completion_audit.md` and the V7 implementation report.

The evaluation script performs a growing-window one-step synthetic comparison from history indices 6–11, runs the planner, and compares its strategies with the deterministic backward

scheduler on the same declared scenarios. It does not hold out independent farms or crop cycles, estimate harvest error, test calibration, or perform statistical significance analysis.

10.2 Tests executed for this documentation audit

The command and outcome from this audit should be read separately from the archived evidence above. On 11 September 2026, the following current-tree command completed with **35 passed in 57.09 seconds**:

```
/tmp/farmtact-docs-venv/bin/python -m pytest -q \  
  tests/numerical/test_planner.py \  
  tests/models/test_explorer_numerics.py \  
  tests/planner/test_research_constraints.py
```

The targeted suite covers fixture determinism and bounds, EWMA setting/hash behavior, planning feasibility and strategy distinction, impossible early sowing, daily mass balance, cost reconciliation, executed-work locks, independent bed/nursery/labour checks, yield semantics, timeout fallback, zero demand, downside aggregation, research reservations, unconfirmed orders, and a same-environment research repeatability check. That repeatability check is not a guarantee that a time-limited solver returns an identical schedule across environments. The suite does not constitute real-world agronomic validation.

11. Limitations and factual gaps

11.1 Scientific limitations

1. There is no fitted growth, maturity, yield, quality, disease, or weather-response model. Scheduled dates and fixed marketable kg/m² are assumptions.
2. No farm-specific crop cycle or harvest outcome has been used for training, calibration, or external validation.
3. Public weather, forecasts, trade records, and News do not affect numerical demand, yield, timing, or prices.
4. The static three-case uncertainty set has no empirical probability or correlation calibration. Its seed does not produce random scenarios.
5. The EWMA is a constant-level forecast. Calling it seasonal would overstate the implementation.
6. The synthetic evaluation's periodic generator favors a smoothing baseline and cannot establish accuracy under promotions, stockouts, trend breaks, buyer churn, or genuine seasonality.
7. A batch's yield changes only through a direct scenario multiplier; a harvest delay does not mechanistically affect mass, quality, shelf life, or later crop development.
8. The visual progress percentage is linear calendar time. It should never be interpreted as percent biomass, maturity, or completion confidence.

11.2 Planning and accounting limitations

1. Decisions are whole-bed Boolean choices. Partial beds, plant-site quantities, continuous area, mixed beds, and harvest-window choices are unavailable.
2. The current farm contract permits at most one existing batch per bed, regardless of whether historical intervals would be non-overlapping. It does not represent a complete sequence of historical/current commitments on one bed.
3. CP-SAT has age buckets but does not explicitly constrain deliveries to FIFO order. The outcome simulator does use FIFO. This can create a difference between the optimized inventory flow and the reported replay, even though expiry penalties usually favor old-stock use.
4. Final-horizon remaining stock is not assigned a terminal salvage value or a general closing-stock penalty in the CP objective. Results can depend on the arbitrary horizon boundary.
5. The objective is not pure expected margin and does not enforce a minimum service target. It combines revenue-like delivery reward with hand-set shortage, expiry, and commitment preferences.
6. Stored scenario weights are not consumed by the solver. They happen to be equal today.
7. “Resilient” is a high-shortage-penalty policy, not robust optimization, CVaR, maximin protection, or a service guarantee under the low-yield case.
8. The solver cash constraint uses a conservative approximation; reported scenario cost uses actual simulated delivery and disposal. These are related but not identical quantities.
9. Demand is fulfilled in aggregate by crop/date. The system does not attribute delivered kg to individual orders, grades, or customers.
10. Residual demand without any booked line in a week has zero selling price. That understates its possible revenue and should remain an explicit missing-price state.
11. Water, energy, HVAC, seed, packaging, buyer-grade, rotation, multi-cut occupancy, supplier lead time, and partner purchases are specified future constraints but absent from the implemented contract and solver.

11.3 Interface and reproducibility gaps

1. The forecast emits harvest mass as `marketable_kg`, while the conversation reference builder looks for `expected_kg` in harvest forecast rows. Batch-level references still expose `expected Marketable_kg`, but the forecast-harvest mass reference is omitted.
2. The backend marks an existing batch ready on and after harvest, whereas the standalone frontend preview makes that bed empty at and after harvest when no accepted allocation overlay is present. Neither view exposes the sanitation interval that still blocks planner occupancy.
3. The preview slider allows the date at `planning_date + horizon_days`, one day beyond the planner's final indexed day (`horizon_days - 1`).
4. The checked-in numerical report contains an `evaluation_scope` field that the current evaluation script does not write. Regenerating the report from the current script would remove that field unless the script is corrected.
5. The historical integrated-demo script still checks for ten crop profiles, while the current catalogue contains twelve. That script cannot report PASS under its current condition even when the current catalogue is correct.

12. Recommended next numerical iteration

The most useful next iteration is to preserve the transparent baseline while replacing implicit assumptions with measured states and testable contracts.

1. **Unify crop-cycle state.** Define one backend state function covering nursery, grow-out, harvest-ready, harvested, sanitation, and available. Return both scheduled and observed states, and make the UI consume it. Keep progress unknown unless there is an explicitly defined estimator.
2. **Collect farm-specific batch outcomes.** Record cultivar, system/zone, sow/transplant dates, repeated observations, harvest window, gross kg, marketable kg, reject reasons, and independent crop-cycle ID. Require enough independent cycles before fitting a challenger.
3. **Build honest harvest baselines first.** Compare recipe median duration/yield, recent farm/system medians, and a partially pooled model using whole-batch temporal holdouts. Report harvest-date MAE, marketable-yield MAE, interval coverage, and sharpness by crop and system.
4. **Strengthen demand evaluation.** Use rolling-origin multi-horizon backtests, cancellation and availability states, buyer/product hierarchy, and explicit missing-price handling. Compare EWMA with last-week and seasonal-naive baselines before considering gradient boosting or quantile models.
5. **Calibrate scenarios.** Estimate residual blocks from independent cycles and demand periods; preserve shared zone/weather shocks. Store effective sample size and realized coverage. Until then, name the current cases “declared stress cases,” not probabilities.
6. **Align optimization and replay.** Enforce FIFO or prove an equivalent inventory formulation, add terminal stock value/cost, consume declared scenario weights, and expose objective terms in physical/financial units. Test the CP objective against the simulator on small enumerated cases.
7. **Make policy semantics contractual.** Add explicit service targets or risk constraints where desired. If Resilient is intended to protect the worst case, implement maximin, CVaR, or scenario-specific service constraints and report the resulting trade-off.
8. **Expand resources only when measured.** Add seed/seedling lots, packing capacity, water/energy/HVAC, grade, packaging, rotation, repeated cuts, and supplier quotes as versioned inputs with units and availability times. Do not insert placeholder capacity as if observed.
9. **Separate observed outcomes from simulated ledgers.** Store actual harvest/delivery records independently, reconcile forecast error versus execution shortfall, and prohibit a simulated worklist from creating actual observations.
10. **Create a reproducible evaluation manifest.** Pin fixture hash, dependency versions, solver parameters, generated report schema, command, timestamp, and git commit. Fail report generation when the archived schema and generator disagree.

Any learned challenger should remain `demo_only` until it beats simple baselines on temporally and biologically independent farm cycles, passes leakage tests, and has uncertainty calibration adequate for its stated use. Even then, the optimizer and its constraints remain separate numerical systems; model accuracy alone does not validate a planting decision.

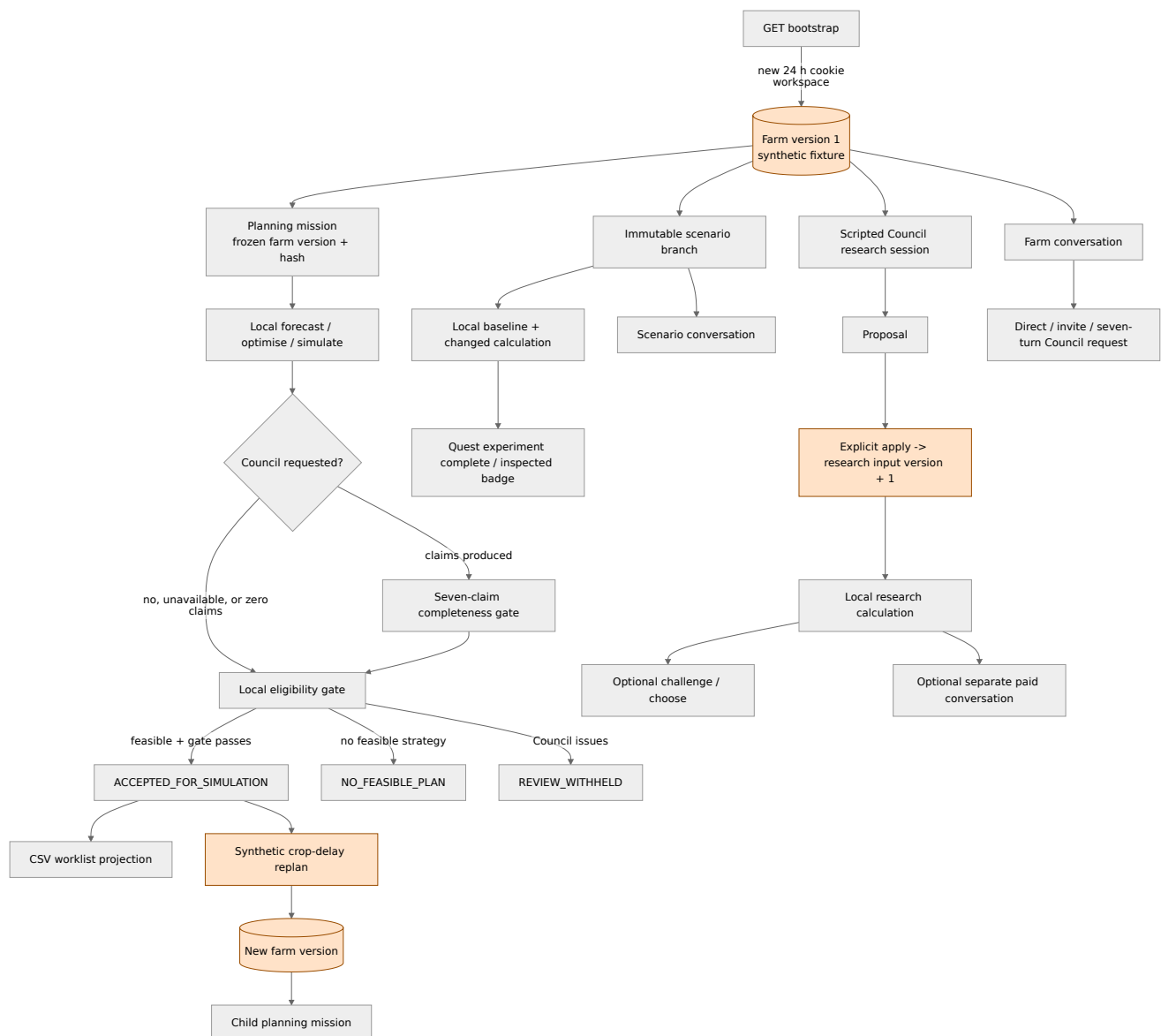
Game backend and state machines

This chapter audits the implemented FarmTact game and simulation backend as of 11 September 2026. It follows persisted state from anonymous workspace creation through planning, automatic simulation acceptance, disruption replanning, scenario branches, quests, conversations and the playable Council research study. It also separates backend facts from browser-derived presentation.

The central finding is simple: FarmTact has a durable **planning and experiment game**, but it does not have a farm-time execution engine. Numerical routines calculate dated ledgers over a frozen horizon. Missions, branches, quest badges and research choices persist workflow progress. None of them advances the farm's `planning_date`, consumes an accepted allocation, changes crop growth stage day by day, fulfils an individual order, or records completion of sowing, transplanting and harvest work. The only implemented main-farm mutation after bootstrap/import is a deliberately synthetic disruption that edits the first batch and creates a new farm version (replan route).

For the numerical equations and their scientific limits, see Numerical models and growth. For provider routing and claim validation, see AI provider and Council. This chapter concentrates on authority, transitions, version binding, concurrency and recovery. It is part of the main scientific implementation report; remediation priorities are consolidated in the next-iteration gap register.

The implemented game in one view



Orange marks the mutable farm/research state and its explicit update transitions. Of those transitions, replan changes the main farm and apply changes only an isolated research input. ACCEPTED_FOR_SIMULATION, the scenario acceptance record, quest progress, a research choice and advisor messages are durable decisions or game progress. They are not agricultural execution. Scenario and research snapshots are isolated copies and never replace the main farm (scenario isolation test, research calculation).

What “simulation” means here

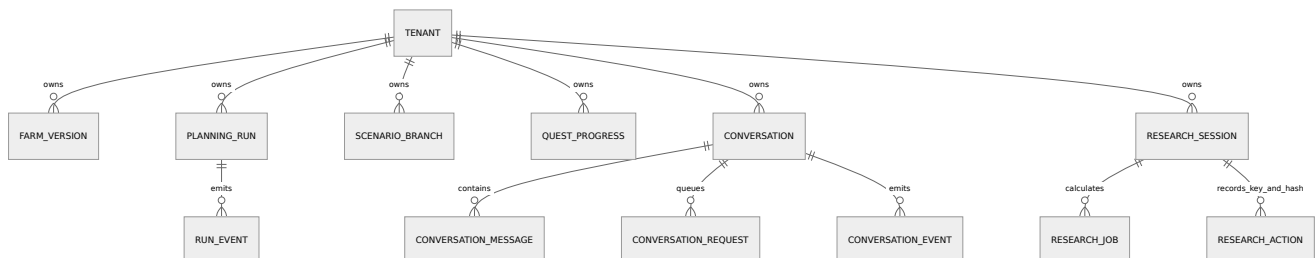
The same word covers several different mechanisms. They should not be presented as interchangeable.

Mechanism	What is actually calculated or persisted	Does farm time advance?	Does it mutate the main farm?
Planner simulation	Daily FIFO inventory ledger and aggregate crop/date demand, harvest, delivery, expiry, stock, cost and margin over the frozen horizon (simulator)	No	No
Automatic mission acceptance	Service records a chosen eligible strategy, hashes, policy version and a copy of its unexecuted allocations as <code>simulated_outcome.work</code> (acceptance)	No	No
Worklist	CSV rendering of the accepted strategy, guarded by current farm version and labelled <code>SIMULATION_ONLY</code> (worklist)	No	No
Replan disruption	Adds seven days and applies a 0.8 yield factor to the first current batch; persists a new farm version and child mission (replan)	No	Yes , but only as a synthetic observation update
Scenario branch	Applies bounded controls to a copy, calculates baseline and branch, and records a simulation-only acceptance if an eligible strategy exists (scenario execution)	No	No
Quest	Records that a matching branch completed and that the user explicitly inspected it; derives two fixed badge labels (quest progress)	No	No
Council research	Applies typed edits to a private study input version, calculates three policies and records an explicit simulation choice (actions)	No	No

Mechanism	What is actually calculated or persisted	Does farm time advance?	Does it mutate the main farm?
Conversation action	Stores a hypothesis_only scenario control proposal after local validation; the user must explicitly create a branch (reply validation)	No	No
Browser mission	Locally screens dated orders against eligible harvest and inventory and stores the selected mission in edition-scoped sessionStorage (derivation)	No	No
Audio	Plays an ephemeral client effect when a result becomes terminal; no sound event is persisted (audio result)	No	No

The planner steps through calendar dates inside one function call. “Day 12” is a row in a hypothetical ledger, not a scheduled backend wake-up. Worker waiting, advisor calls, scripted dialogue and UI animation do not change the biological calendar.

Persistence and ownership



The session cookie maps to a tenant through a SHA-256 hash. Authentication accepts only tenants created during the previous 24 hours; the cookie is HTTP-only, same-site strict and secure on HTTPS (session store, bootstrap). Rows themselves are not deleted when authentication expires.

Farm versions are append-only by (tenant_id, version). Version allocation locks the tenant row, calculates $\max(\text{version})+1$, hashes the whole payload and inserts a new row (farm persistence). Planning runs, scenario branches, conversations and research sessions then freeze full JSON snapshots. Most relationships also have tenant-aware foreign keys; conversation messages, requests and events have their own monotonically increasing sequences (conversation schema).

There is no domain table for executed tasks, crop-stage observations, inventory movements, order fulfilment, simulated clock ticks or game turns. work is nested JSON in a run, and quest progress is a compact per-tenant/per-quest JSON record.

HTTP surface and transition authority

All routes below require the tenant session except bootstrap and the fixed health endpoint. Mutating requests pass same-origin, body-size and admission checks before route parsing (middleware, admission).

Main farm and planning missions

Method and route	Transition / result	Guard and idempotency
GET /api/v1/bootstrap	Creates tenant + synthetic farm v1 when no valid cookie; otherwise returns latest farm view and latest run	New-session IP/global limits; cookie lifetime 24 h
POST /api/v1/imports	Validates exactly one fixture or Farm, always appends a farm version; after a terminal latest run also creates a local refresh mission	No idempotency key ; tenant transaction serializes the version
POST /api/v1/planning-runs	Freezes latest farm and creates CREATED mission	Required key, tenant-wide key/hash equality, and at most one CREATED/RUNNING planning run
GET /api/v1/planning-runs/{id}	Returns public run and current acceptance_stale flag	Tenant ownership
GET /api/v1/planning-runs/{id}/events	SSE after query/header cursor; stops after terminal run	Per-IP open and active-stream limits
POST /api/v1/planning-runs/{id}/cancel	Sets cancel_requested only while created/running	Cooperative; response always says cancel_requested even if already terminal
GET /api/v1/planning-runs/{id}/replay	Returns stored run/events with replay labels	Run must be terminal; no inference
POST /api/v1/planning-runs/{id}/replan	Writes the fixed synthetic first-batch disruption, new farm version and child mission	Parent must be accepted, current farm version must equal parent input, required tenant-wide run key; body/key conflict is 409
POST /api/v1/strategies/{id}/accept-for-simulation	Always 403	Browser cannot accept; backend policy owns acceptance
GET /api/v1/planning-runs/{id}/worklist.csv	Renders accepted allocation list	Accepted strategy must exist and latest farm version must still equal run input version
GET /api/v1/reports/{id}	Alias of public run	Tenant ownership

Method and route	Transition / result	Guard and idempotency
GET /api/v1/strategies/{id}	Searches all owned run JSON for the strategy	Tenant ownership; no run/version parameter

The definitions are in the main route installer. Import's lack of idempotency is especially relevant: an identical retry creates a new version, makes accepted worklists stale and may enqueue another refresh.

Scenario branches and quests

Method and route	Transition / result	Guard and idempotency
GET /api/v1/scenarios	Returns every owned full branch, newest first in Python	No pagination or branch count limit
POST /api/v1/scenarios	Freezes one source and creates DRAFT branch	Required tenant-wide key; source is explorer or parent/conversation; completed parent required; quest must change its named assumption
POST /api/v1/scenarios/{id}/run	DRAFT -> QUEUED; later worker claims RUNNING	First run_key binds the branch forever; same key reuses, any other key gets 409
GET /api/v1/scenarios/{id}	Returns full saved branch/results	Tenant ownership
GET /api/v1/scenarios/compare?ids=...	Returns one to three completed branches and shared baseline	Distinct IDs; same comparison root, baseline hash, baseline forecast settings and calculation version
GET /api/v1/quests	Combines four fixed quest definitions with progress rows	No mutation
POST /api/v1/quests/{id}/inspect	Adds branch ID to inspected set, marks quest completed and grants fixed badges	Scenario must be completed and have the matching quest ID; semantically deduplicated, no request key

See scenario request and controls and scenario routes. Controls are bounded: batch delay 0..14 days, marketable yield 50..100%, demand, labour and cash 50..150%. Delay/yield require a known batch; changed demand requires one of the four fixture recipe crops. Demand control scales both historical ordered kg and all booked/cancelled order quantities for that crop (control application). A child starts from its parent's already-modified input, so percentages compound across branches.

Persistent conversations

Method and route	Transition / result	Guard and idempotency
GET /api/v1/conversations/advisors	Returns current seven-person roster	Read only

Method and route	Transition / result	Guard and idempotency
GET /api/v1/conversations	Lists public conversation metadata without messages	Tenant ownership; no pagination
POST /api/v1/conversations	Freezes farm, completed scenario or current completed research calculation; creates OPEN	Required tenant-wide conversation key; same body reuses
GET /api/v1/conversations/{id}	Public state, all messages, frozen tool/evidence context	Tenant ownership
GET /api/v1/conversations/{id}/replay	Same messages with replay labels	Read only, no provider call
POST .../{id}/messages	Appends user message + direct one-role QUEUED request atomically	Required key scoped to conversation; one active request per conversation
POST .../{id}/invite	Queues invited role followed by author of referenced advisor message	Must reply to an advisor and invite a different advisor
POST .../{id}/council	Queues seven ordered roles	Optional reply must belong to same conversation
GET .../{id}/events	JSON cursor page or SSE	Cursor nonnegative; stream stops when latest request is terminal

The route implementation is in `conversations`. A farm conversation always freezes the latest farm; its optional `snapshot_id` checks the farm ID but cannot select an old farm version. Scenario conversations require `COMPLETED`. Research conversations require a completed result whose version is the current research input version, although the error text says “complete and select” and the implementation does not require chosen (snapshot freezer).

Conversation request admission counts one user message plus all requested roles against a hard 120-message conversation limit. User message and job creation are one locked transaction, with (conversation, tenant, key) uniqueness (request persistence).

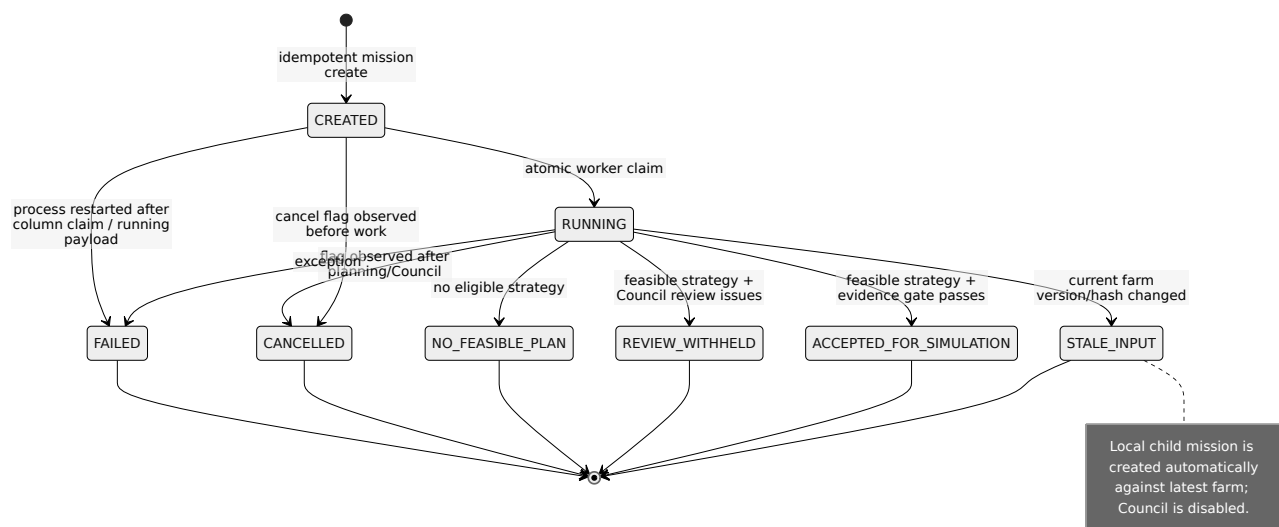
Playable Council research

Method and route	Transition / result	Guard and idempotency
GET /api/v1/council-research	Lists up to 30 session summaries	Query limit occurs before Python sort; no explicit SQL order
POST /api/v1/council-research	Creates isolated synthetic study v1	Required tenant-wide session key; 30-session tenant cap
GET /api/v1/council-research/report	Returns bounded local report excerpts, fixed citations and screenshot URLs	Read only; paths are fixed in code

Method and route	Transition / result	Guard and idempotency
GET /api/v1/council-research/{id}	Returns complete mutable session JSON	Tenant ownership
POST .../{id}/actions	Applies one of 12 action types and normally increments revision	Required key unique across all research actions in the tenant ; body revision must equal current; 400 actions/session

The route implementation and exact action switch are in Council research routes. say, select, propose, apply, discard, challenge, resolve, choose, configure, stop, next and run are one endpoint because the request model is a tagged action with optional fields. Unknown action names and extra fields fail schema validation; action-specific guards validate required context. Known optional fields are not a fully discriminated per-action schema and may be unused on a different action (action schema).

Main planning state machine



Creation freezes the whole farm payload and its hash, source cards, evidence file hash, market summary, news context, Council version and parent/disruption metadata before queueing (mission creation). Store.create_run locks the tenant, reuses an equal key/hash, rejects a changed body under the same key, and permits only one active planning mission for the tenant (run persistence).

The worker calculates all three strategies locally first. Strategy eligibility is exactly:

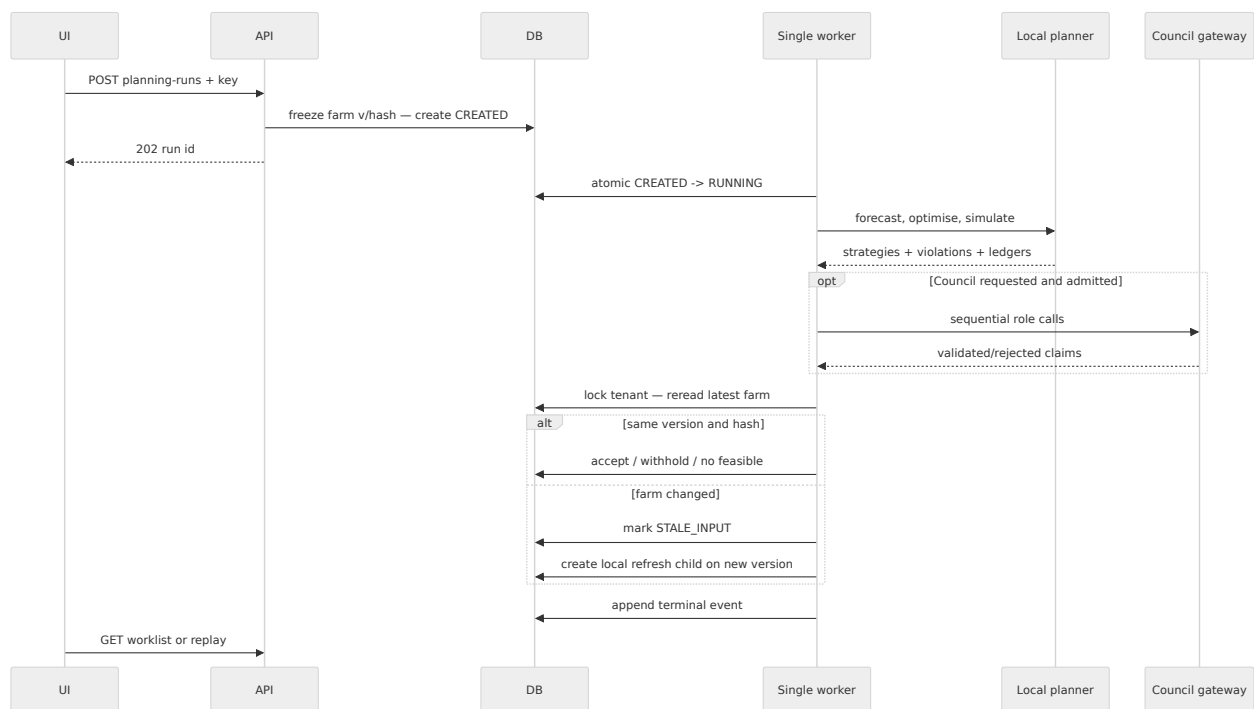
1. stored strategy status == FEASIBLE;
2. a fresh validate_allocations(frozen_farm, allocations) returns no violations;
3. choose Balanced when present, otherwise the first eligible strategy in the planner's current list order;
4. if Council claims exist, all seven roles must appear in exact roster order, all must have status=validated, and all must recommend proceed_simulation (Council gate);

- immediately before acceptance, the latest farm must still have the same version **and** content hash.

An accepted record binds input version/hash, strategy ID/hash, validation report hash, policy version, service actor and `simulation_only=true`. The public manual acceptance endpoint always returns 403, so no browser or advisor can bypass that gate.

There is a significant asymmetric rule. If a requested Council fails or is blocked before producing any claim, `claims=[]`; the evidence gate uses numerical-baseline and the strategy can still be accepted. If even one claim was persisted before failure, the incomplete seven-role set produces review issues and acceptance is withheld. This is intentional in the current test contract (mission gate test), but the API has no explicit `council_required_for_acceptance` policy field.

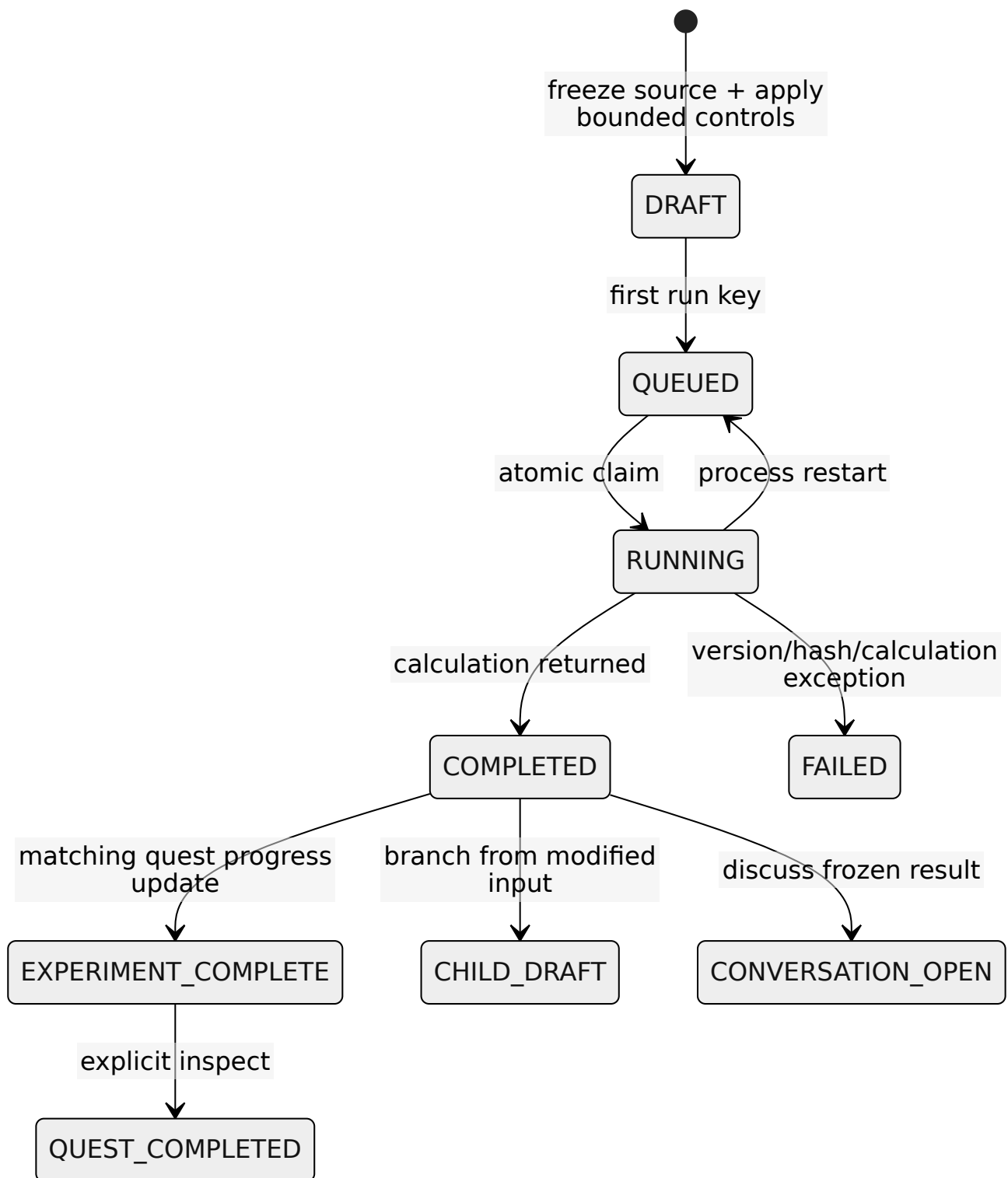
Planning sequence, stale input and replan



Stale-input refresh intentionally disables the Council, avoiding automatic repeat paid inference. Process restart is different: a planning job recorded as running becomes FAILED and is never replayed because the system cannot know whether a paid request completed (restart rule).

Replan is only available from `ACCEPTED_FOR_SIMULATION`. It requires the parent farm version still be current, preserves existing sow/transplant dates, changes the first batch's expected harvest date/yield, appends a farm version and creates a child. Its request schema exposes only `crop_delay`; callers cannot choose the batch, delay or yield factor (schemas). This is a fixed demo transition rather than a general disruption engine.

Scenario branches, comparisons and quests



A scenario source is one of:

- a saved explorer snapshot;
- a completed parent scenario's modified input;
- a conversation's frozen snapshot, with a scenario conversation resolving back to its scenario parent;
- otherwise the current farm.

The branch stores its modified input, root baseline, hashes, optional generator and forecast settings, news context and calculation version. The worker refuses to substitute a current forecast/planner implementation when the frozen version does not equal the installed version (version check). On success it calculates same-policy baseline/branch deltas, validates allocations again and writes its own simulation-only acceptance. A COMPLETED branch may still have `simulation_status=NO_FEASIBLE_PLAN`; completion means the experiment calculated, not that the requested farm plan was feasible.

Comparison eligibility is strict enough to prevent mixing unrelated roots: one to three distinct, completed branches must share comparison root, baseline hash, baseline forecast configuration and calculation version. Child control factors apply to the parent's already changed snapshot. For example, a 90% cash child of a 90% cash parent represents 81% of the root cash, not another branch at 90% of root.

Quest state has no prerequisite graph or scoring engine. Four definitions are hard-coded. Creating a matching changed branch is permitted in any order; worker completion adds it to `experiment_ids`; explicit inspect adds it to `inspected_ids`, changes status, and assigns Experiment explorer plus Tradeoff discovered. Even infeasible results count as completed learning (infeasible quest test).

`affected_bed_ids` compares allocation hashes grouped by bed across every policy. `affected_deliveries` identifies changed order inputs or orders on dates where aggregate ledger demand/delivery changed. It explicitly is not per-order fulfilment attribution (impact calculation).

How the three Councils work

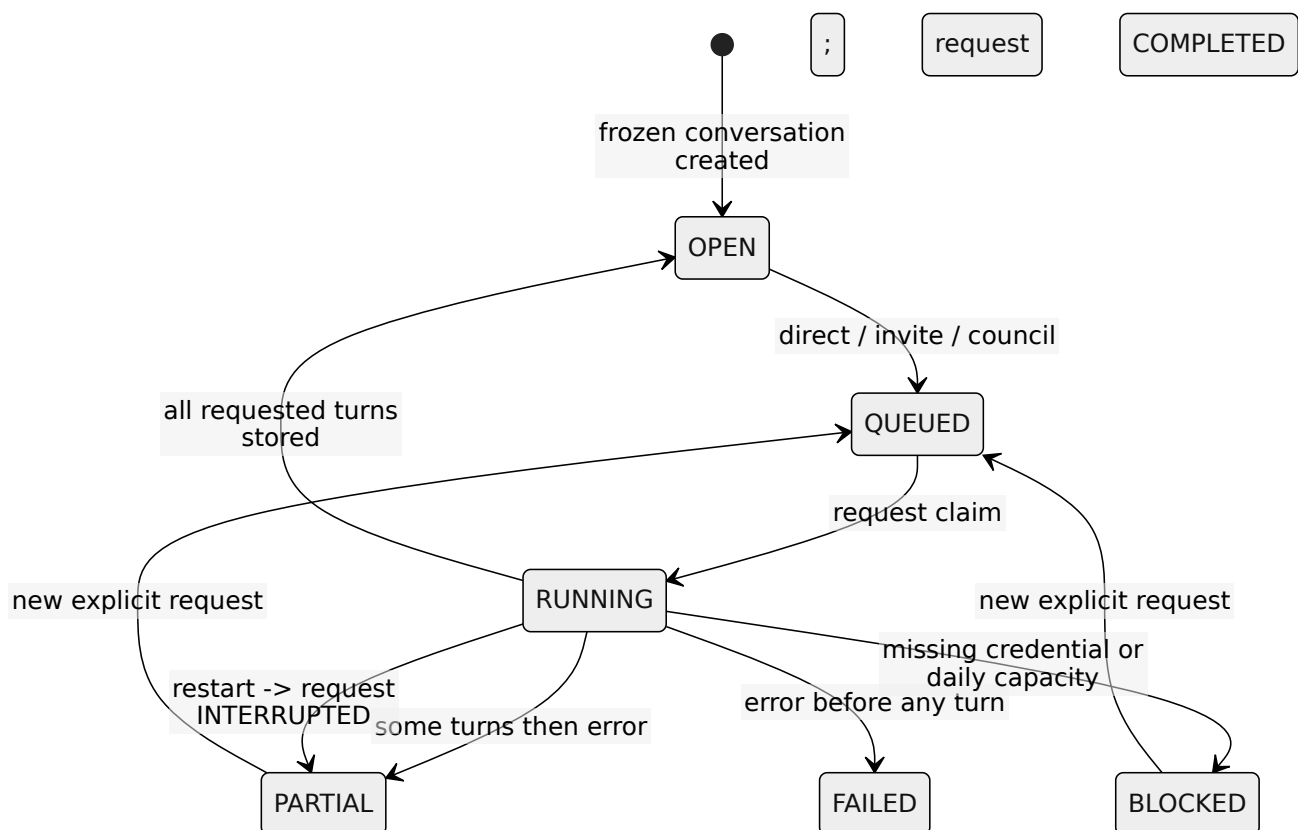
1. Mission Council: seven provider-backed findings

The current roster is Ravi/demand, Hana/weather, Idris/market, Mei/production, Lina/supply chain, Ben/profit and Asha/planning chair. The mission Council runs one structured provider request per role with at most two shared schema repairs. Each role receives the same frozen numerical strategy summary and bounded reference map. Only the planning chair receives the claims already produced by earlier roles (Council loop).

This is sequential multi-role review, not a deliberative debate. Specialists do not see or rebut other specialists. There are no implemented challenge rounds, votes, tool calls initiated by roles, or user intervention points. The chair synthesizes prior claims once. Local validation checks schema, role, known reference membership and literal numeric correspondence; it does not prove that qualitative prose follows from the cited item.

Council output may block automatic simulation acceptance but cannot alter a strategy or execute work. All seven findings must validate and recommend proceed. Any dissent, rejection, duplicate, reorder or partial set with at least one claim withholds acceptance.

2. Conversation Council: a persisted reply chain

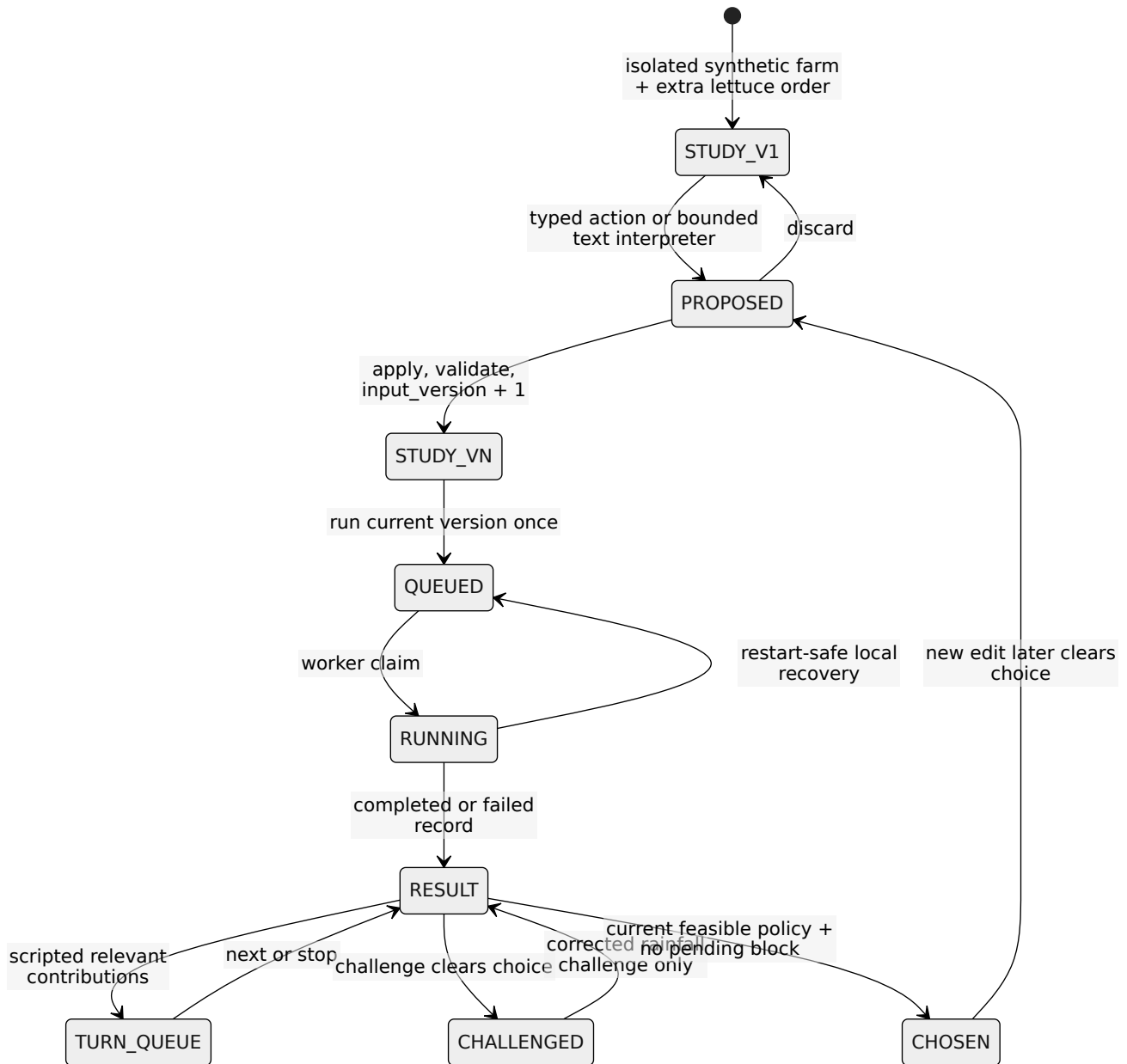


Direct mode has one role. Invite mode has two: the invited advisor responds to a specific advisor message, then that original advisor responds to the invited reply. Council mode produces seven turns in roster order; each turn after the first replies to the immediately preceding generated message. The planning chair must use the final conclusion relationship to be marked a planner conclusion (conversation execution).

Every generated message is stored before the next provider call, so a later failure preserves a useful partial transcript. Restart marks uncertain running requests INTERRUPTED, conversation PARTIAL, and never automatically retries possibly billed work (conversation recovery). A new explicit request is allowed after a terminal partial/failed/blocked request.

COMPLETED is transport/workflow completion. A reply can be stored with `validation_status=unsupported`, blocked proposed actions and validation errors, yet the request still becomes COMPLETED. Thus conversation status does not mean that the Council's conclusion passed a decision gate. Conversation proposals are always hypotheses and require an explicit user-created scenario branch.

3. Council research: scripted study and local tool



The study begins from a new synthetic fixture plus an 18 kg lettuce order due in 48 days. It records `dialogue_mode=scripted_research`, `inference_calls=0` and `operational_execution=false` (session creation). Its text interpreter recognizes a small set of reservation, order-status, labour and challenge phrases. It never executes free text. A proposal has a `base_version`; only explicit apply validates the controls and creates the next research input version (proposal/application).

Each input version can have only one research job. One queued/running research job is allowed per tenant across all studies, and one study stores at most 12 result versions. A restart requeues a running job because calculation is local and has no paid side effect. When a later version calculates before v1 has a completed baseline, the worker reconstructs v1 from the original controls. A result that finishes after inputs changed is archived and cannot overwrite current input state (research worker).

After a result, the backend creates fixed scripted contributions only for control categories that changed, plus a planner message. continuous and checkpoints change how pending_turns are

revealed; they do not change numerical logic. The presentation concept, selection mode and animation setting are also game/UI configuration only.

A challenge stores one mutable object and clears any choice. Rainfall gets a specific supported correction: outdoor rainfall is context, not a sheltered hydroponic yield input. Other challenge categories have no verified correction. resolve=evidence leaves status unchanged; corrected is allowed only for rainfall; unresolved and reject continue to block choose. choose requires a completed result for the current version, a feasible violation-free selected policy, no proposal, and no challenge whose status differs from corrected (challenge, choose). It writes a simulation-only research choice and never affects the main farm.

The optional “actual DeepSeek interpretation” in this screen creates a separate research-snapshot conversation and sends a direct one-role message. It is not the scripted dialogue and it is not a full Council (UI boundary).

Worker concurrency, admission and recovery

One daemon thread polls all queues every 300 ms. In each pass it exhausts pending planning runs, then scenarios, research calculations and conversations, in that fixed order. Each job runs synchronously on the same thread (worker loop). This provides simple atomic claiming but no fairness, parallelism, per-tenant scheduling or isolation between paid provider latency and local numerical play.

The application-level concurrency rules are:

Work type	Active limit	Claim rule	Restart behavior
Planning mission	One CREATED/RUNNING per tenant	Conditional status update	Running -> FAILED; explicit new mission required
Scenario	No tenant-wide active cap	Conditional QUEUED -> RUNNING	Running -> QUEUED
Research calculation	One queued/running per tenant	Conditional job status update	Running -> QUEUED
Conversation	One queued/running request per conversation	Locked request row	Running -> INTERRUPTED; conversation -> PARTIAL
Provider calls	One at a time inside each synchronous job	Gateway/run budget	Never automatically repeated after uncertain planning/conversation interruption

Planning and conversation inference attempts have durable request admission: 6/IP/minute, 20/IP/hour and 12/tenant/hour. All writes also consume 60/IP/minute; all API requests consume 600/IP/minute and 3,000 global/minute. The shared provider reservation ceiling is 48 calls per UTC day. SSE openings have their own 12/IP/minute rule plus four active streams/IP, 32 globally and a 360-second deadline (rules, stream slots). Scenario and research calculation actions do not consume the AI-specific admission bucket because they make no provider calls.

Cancellation exists only for main planning. It is cooperative: a queued mission will stop before planning; a running mission checks after local planning and the Council receives a callback it can check between bounded work. There is no external kill of a current solver or provider HTTP request. Scenarios, research jobs and conversations have no cancel route. Research stop clears pending scripted turns but explicitly does not cancel a calculation.

Browser-derived game state and audio

The UI derives several useful views that are not backend domain state:

- `DecisionMission` groups future booked orders by crop/date, subtracts harvests and inventory that appear eligible by date/shelf life, and selects the earliest positive provisional gap. It warns that it does not allocate those lots across earlier orders. It is edition-scoped `sessionStorage`, not a server entity.
- `NumericalPerspectives` renders seven named perspectives deterministically from scenario comparison metrics and violations. These cards are not provider messages even though they use the same advisor personas.
- selected scenario, explorer filters and current research/actual-conversation pointers live in edition-scoped session storage. The durable objects remain in the backend; the selection/navigation state does not.
- main mission events use SSE plus 2.5-second polling fallback. Scenario and research screens poll. Conversation supports JSON cursors and SSE.

Audio defaults are edition-specific and preferences are stored in local storage, but every visit begins inactive until direct user activation. Background music pauses while hidden. Effects are rate-limited in memory and stop each other so only one plays at a time (audio state, playback).

Main-run sound outcome is derived from terminal status and the accepted, Balanced or first strategy's `shortfall_kg`; scenario sound uses the same fallback order (sound mapping). A run only becomes audible if this browser instance started/replanned it and kept its ID in an in-memory set. `playSimulationResult` suppresses duplicate sounds in that page lifetime. These cues are presentation feedback, not durable evidence that a backend event was consumed, and are appropriately absent from replay provenance.

Gaps and acceptance tests

The IDs below are stable references for remediation. Severity reflects decision integrity and reliability, not visual polish.

ID	Severity	Implemented gap and consequence	Recommended acceptance test
GAME-01	Critical	There is no simulated clock or execution state machine. Acceptance and worklist export cannot record task start/completion, inventory movement, batch stage, order fulfilment or observed outcome. Gamified progress can therefore be mistaken for farm progression.	Given an accepted version, advance a synthetic clock through sow/transplant/harvest/delivery; assert idempotent task events update new versioned state, conserve mass, never touch operational integrations, and replay exactly. Until built, assert every UI/API result retains <code>simulation_only</code> and no “completed farm work” wording.
GAME-02	High	One synchronous worker causes head-of-line blocking across all tenants and job types. A slow Council/provider request can delay unrelated scenario and research calculations for up to its bounded wall time.	Queue a blocked 300-second provider job and local jobs from another tenant; require bounded local-job start latency and independent worker pools/leases. Kill a worker mid-claim and prove each job class follows its documented recovery rule once.
GAME-03	High	Requested Council acceptance is asymmetric: zero claims after block/failure passes as numerical baseline, while one-to-six claims withhold. The request has no field stating whether Council is advisory or required.	Parameterize missing credential, budget block, failure before first claim, failure after each claim and seven valid claims. Assert the chosen explicit policy (advisory or required) and persist it in the acceptance record.
GAME-04	High	Neither provider-backed Council implements the build specification's two challenge rounds. Mission specialists are independent; only the chair sees prior findings. Conversation Council is a linear reply chain. Scripted research has one user challenge object but no agent debate.	Record the exact expected topology: participant visibility, round number, challenge target, rebuttal and final synthesis. Test request ceilings and restart behavior for every round, or change the specification/UI to “seven sequential reviews plus chair synthesis.”

ID	Severity	Implemented gap and consequence	Recommended acceptance test
GAME-05	High	Research challenge version binding is incomplete. challenge.input_version is recorded, but apply leaves the challenge in place and resolve/choose do not require its version to match current inputs. A correction to an old version can authorize choosing a later version. A new challenge also overwrites the prior one.	Challenge v1, apply v2, resolve the old challenge and attempt choose v2; require rejection or explicit challenge migration/revalidation. Preserve multiple challenge records with version and resolution evidence.
GAME-06	High	API idempotency exists for most expensive transitions, but browser helpers generate a fresh UUID on every call and do not retain it across an uncertain network retry. Import has no key at all. Duplicate imports append versions and stale accepted work; a lost mission response can be retried as a second run after the first finishes.	Simulate “server commits, response lost” for import, mission, scenario creation/run, conversation send and research action. Reload/retry from UI and assert exactly one logical mutation/provider job using a persisted operation key.
GAME-07	High	Conversation request COMPLETED only means all turns were stored. Every reply, including the final chair, may be unsupported; the conversation returns to OPEN and looks terminal-successful. No separate Council validity/result state exists.	Make one and all seven replies fail semantic/reference validation. Assert transport status and decision-validity status are distinct, the final result is visibly withheld, and no proposed action can be applied implicitly.
GAME-08	High	The scenario “run once” key is permanent. A failed calculation cannot be retried on the same immutable branch, including after a transient local failure. The UI must create a new branch, losing a direct retry/recovery relation.	Inject a transient calculation failure after claim; retry same run key and a dedicated retry endpoint. Require either safe same-input local recomputation with attempt history or a child retry that explicitly links to the failed branch.

ID	Severity	Implemented gap and consequence	Recommended acceptance test
GAME-09	Medium	Frozen calculation version support is equality with the currently installed code, not a runner registry/container reference. Old uncalculated branches or jobs fail after deployment; exact source commit is not embedded in every game object.	Create DRAFT/QUEUED objects on version N, deploy N+1, then prove either version-N execution remains available or a migration creates a separately identified result without overwriting frozen provenance.
GAME-10	Medium	Research messages and events are mutable JSON arrays truncated to 120 and 240. The separate action table retains only key/hash identity, not action payload, response, resulting revision or challenge history. Long studies lose early audit detail.	Execute 400 actions and reconstruct every state transition from append-only records. Assert old messages/events remain queryable with pagination and action retries return the original response revision.
GAME-11	Medium	Scenario storage/listing has no per-tenant branch cap or pagination. Conversation listing also lacks pagination. Expired tenant data is not reclaimed by the application. A 24-hour anonymous session can create substantial persistent JSON and every scenario list returns it all.	Fill to documented quotas, verify 429 before expensive work, paginate with stable cursors, expire a tenant and prove retention/deletion policy without affecting active edition data.
GAME-12	Medium	Only planning can be cancelled, and its cancellation cannot interrupt current solver/provider work. Scenario, research and conversation jobs lack cancel endpoints.	Cancel before claim, during solver, between Council turns and during provider I/O for every job type. Assert bounded terminal latency, no new paid request after cancel, unused reservation reconciliation and unambiguous terminal status.

ID	Severity	Implemented gap and consequence	Recommended acceptance test
GAME-13	Medium	Main replan is hard-coded to the first batch, +7 days and $\times 0.8$ yield. The request's disruption enum does not bind a target or magnitude, so it is a demo rather than a general game action.	Submit typed disruptions for explicit batch/version with bounds; reject stale/unknown targets; prove executed commitments remain identical; hash the observation and deterministic child input.
GAME-14	Medium	Strategy fallback is Balanced, then first eligible in current list order. No explicit ranking result explains why the fallback won if Balanced is absent.	Permute strategy order with Balanced infeasible; require an explicit deterministic rank/tie-break record and stable selected ID across permutations.
GAME-15	Medium	Scenario affected deliveries and the planner ledger are aggregate crop/date flows, not per-order allocations. Quest completion can establish that tradeoffs were inspected, but not which buyer order is covered.	Add two same-crop orders on the same/earlier dates and competing lots. Require an allocation ledger with lot-to-order quantities, expiry, shortfall and mass balance before displaying order-level coverage.
GAME-16	Medium	Browser mission, persona perspectives, navigation pointers and sound completion memory are local derived state. They are not revisioned backend progress and do not synchronize across tabs/devices; sound does not acknowledge event consumption.	Change farm/branch in another tab and verify derived mission invalidates by hash. Reload during a terminal event and assert UI truth stays correct with or without sound. Label deterministic persona cards separately from actual Council messages.
GAME-17	Low	Cancel returns <code>{"status": "cancel_request"}</code> for terminal runs even though it changed nothing. This can mislead client state.	Cancel every terminal status and require an explicit no-op/409 plus unchanged persisted record; cancel active run and return the actual current cancellation state.

ID	Severity	Implemented gap and consequence	Recommended acceptance test
GAME-18	Medium	One conversation contract test assumes cached public source D04 is present, while <code>source_views()</code> legitimately returns an empty list when the ignored public cache is absent. A fresh offline checkout therefore fails a game test for environmental data availability rather than conversation behavior.	Seed a minimal versioned public-context fixture in the test or assert a typed unavailable source state. Run the same test from a clean Git checkout with network disabled and with a populated cache; conversation-state assertions must be identical.

Specification corrections to keep synchronized

These observations refer to the pre-audit specifications. The documentation update now incorporates the corrections; GAME implementation gaps remain future work.

These are documentation corrections, not claims that the code should necessarily grow to match every older requirement.

1. Build specification lines 527, 717 and 833 describe two challenge rounds. Current mission and conversation Councils have no challenge-round state. Specify seven sequential reviews and a final chair synthesis, or design the missing versioned rounds.
2. Build specification line 523 lists terminal states including `MISSING_INPUT`, `SOURCE_UNAVAILABLE` and `BUDGET_EXCEEDED`; the planning-run state machine uses warnings/council status for those conditions and terminal run states `STALE_INPUT` and `ACCEPTED_FOR_SIMULATION`, which that list omits.
3. Build specification lines 630 and 693, and DeepSeek runtime specification DS-20, refer to idempotent simulated work. The implementation has idempotent plan/branch/job creation and a worklist, but no simulated work execution API or work-event table. Narrow the claim or implement GAME-01.
4. Build specification line 735 calls the CSV a practical worklist. Clarify that it is a stale-version-guarded simulation projection with no completion/update path.
5. Current product copy should reserve “Council” qualifiers: **mission Council** is seven provider calls, **conversation Council** is a seven-turn provider chain, and **Council research** is scripted with zero provider calls unless the user opens and sends the separate optional actual-advisor question.
6. “Plant growth simulation” should mean dated yield endpoints and inventory flow unless and until GAME-01 and an empirical growth model exist. No backend transition models daily morphology, environment response or stage observation.

Reproducible verification boundary

The existing automated suite covers mission acceptance/replay/replan and stale worklists (security API tests), scenario isolation, branching, comparison, quests and restart recovery (scenario tests), conversation idempotency, reply graphs, validation, partial failure, tenant isolation and PostgreSQL persistence (conversation tests), and research controls/session behavior (research tests). PostgreSQL tests also cover concurrent planning/replan idempotency and event sequencing (concurrency tests).

The coordinated fresh offline run used:

```
pytest tests/council_research/test_sessions.py \  
  tests/council_research/test_independent_review.py \  
  tests/gameplay/test_scenarios.py \  
  tests/gameplay/test_conversations.py -k 'not real_postgres'
```

It reported **66 passed, 1 failed, 1 deselected in 36.06 seconds**. The failure was `test_seven_current_advisors_and_frozen_create_idempotency` at its assertion that `source:D04.summary` exists. The ignored public-data cache was absent; no source fetch was run. This is GAME-18, not evidence that conversation idempotency failed. No database service, external provider or inference was started for this chapter.

The highest-value next verification is a deterministic lifecycle test harness that queues all four job classes, injects response loss and process restarts at each claim/commit boundary, and proves the state diagrams above. That would validate workflow correctness. It would not establish agronomic validity, real farm performance or safe operational execution.

System, data, and evidence architecture

Audit date: 11 September 2026 UTC

Repository baseline: commit a96025e (the working tree may contain the documentation changes prepared for the next release)

Published application covered: immutable v7, with v1–v6 retained as separate editions

This chapter describes what the FarmTact source and recorded release evidence implement. It does not promote roadmap items in the v1.2 specification to shipped capabilities. FarmTact is a synthetic planning demonstration: its farm records, recipes, demand histories, schedules, yields, prices, costs, scenarios, and simulated outcomes are fictional. Real public weather, forecast, trade, news, and scholarly records supply attributed context. They do **not** automatically change demand, yield, recipe, price, or solver inputs.

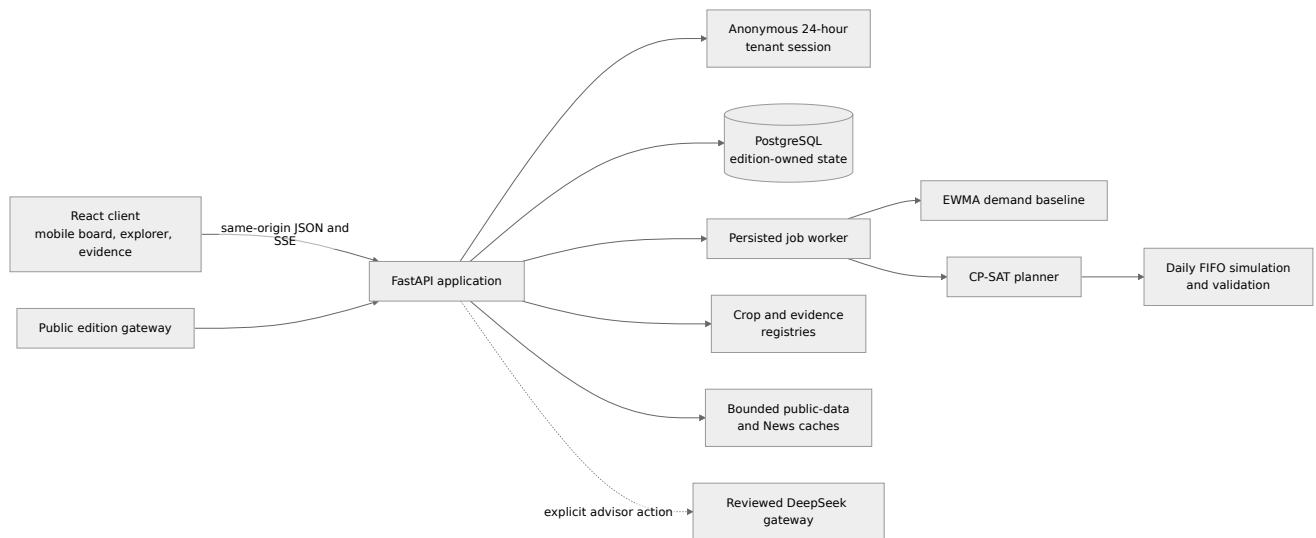
The key separation is therefore:

Real public information can help a person or advisor ask a better question. The numerical planner only changes when a validated, tenant-owned farm snapshot or an explicit bounded scenario changes.

The implementation evidence for that statement is the empty `public_features_used` array in the committed feature manifest, the planner's Farm-only input in `engine.py`, the News context's explicit `numerical_effect="none"` in `news.py`, and the bounded scenario controls in `scenarios.py`.

Implemented system

FarmTact is a React/TypeScript client served by a Python FastAPI application. PostgreSQL stores anonymous tenant state, immutable farm versions, planning jobs, scenario branches, conversations, explorer snapshots, and research sessions. A small in-process worker claims persisted jobs. Local Python code performs forecasting, constraint solving, simulation, comparison, and validation. The DeepSeek boundary is optional for explicit advisor interpretation and is not the source of numerical values.



Primary implementation links:

- Application assembly, lifecycle, API boundaries, worker, and bootstrap: `services/api/app.py`
- Persistent core store and tenant predicates: `services/api/store.py`
- Browser API adapter: `apps/web/src/lib/api.ts`
- Board and farm-world views: `Board.tsx`, `World.tsx`
- Data, crop, and evidence views: `DataExplorer.tsx`, `PublicExplorer.tsx`, `Rooms.tsx`
- Public API response contracts: `services/api/view_contracts.py`

The client uses edition-scoped browser storage for presentation preferences and continuation pointers such as the selected scenario or explorer snapshot, the current research/advisor identifiers, and a cached decision-journey view. Authoritative farm, plan, scenario, conversation, and research records remain server-side and are reloaded through owned APIs. API calls use a random session cookie; the database stores its SHA-256 hash rather than the bearer value. The session is an anonymous workspace identity, not a verified user account.

Data domains and authority

Domain	Current contents	Origin	Can directly drive the planner?	Persistence and authority
Private farm snapshot	16 beds, four recipes, batches, orders, demand history, opening inventory, labour, nursery capacity, and cash	Synthetic fixture or schema-valid JSON import; current contract still restricts all records to <code>synthetic_demo</code>	Yes	Versioned per tenant in <code>farm_versions</code> ; this is the planner's numerical authority
Derived demand and harvest baseline	Confirmed orders, residual weekly EWMA, scheduled harvests	Deterministic local calculation over the frozen farm snapshot	Yes	Frozen into planning, explorer, and scenario results with model/configuration/input hashes

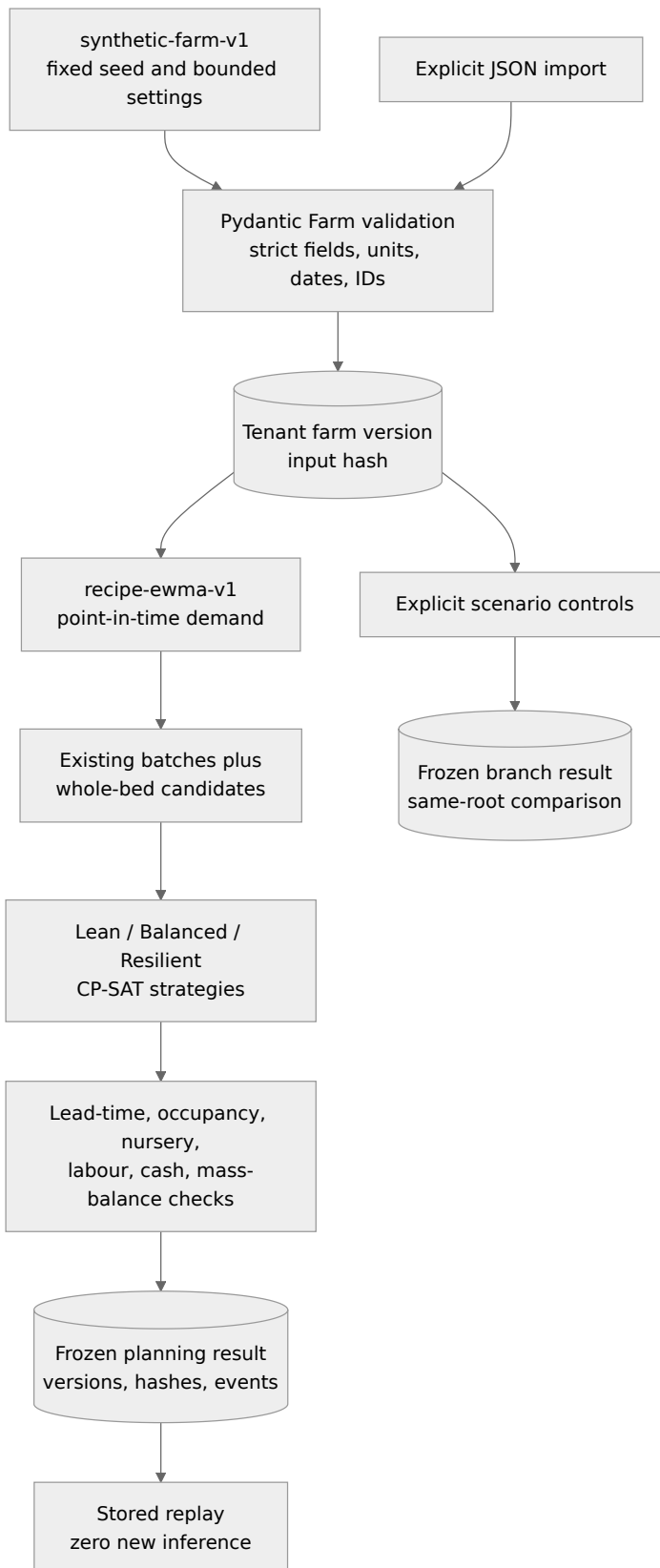
Domain	Current contents	Origin	Can directly drive the planner?	Persistence and authority
Plan and simulation	Lean, Balanced, Resilient allocations; daily inventory ledger; scenario metrics	Deterministic CP-SAT plus local simulation	Yes	Persisted with calculation version, strategy IDs, constraints, hashes, and simulation-only acceptance
Scenario/playground	Delay, yield, demand, labour, cash, bed-reservation, order-status, and EWMA-alpha changes	Explicit bounded synthetic controls	Yes, inside the frozen branch only	Tenant-owned immutable roots and versioned results; no automatic main-farm mutation
Public observations	NEA station weather, NEA forecasts, SingStat trade volume, NASA POWER history	Real public APIs	No	Content-addressed raw snapshots and normalized cache; displayed with source/time/licence limits
News	RSS metadata from four allowlisted feeds; three additional sources visibly unconnected or blocked	Real public headline metadata	No	Separate bounded cache; copied into a decision snapshot at its cutoff
Crop knowledge	Twelve crop concepts linked to 24 reviewed publication records	Curated public/government and scholarly metadata	No	Version-controlled JSON; recipe parameters remain separate synthetic engineering assumptions
Community/market reactions	Validated supplied-record adapter with no connected feed	No runtime records in the published demonstration	No	Status is <code>not_connected</code> ; it cannot manufacture demand or sentiment
AI advisor prose	Qualitative interpretation of frozen numerical and evidence references	Actual DeepSeek only when explicitly invoked; scripted v7 dialogue otherwise	No	Persisted and locally validated; unsupported text cannot authorize a change

The private contract in `contracts.py` is intentionally narrower than the future operational schema proposed by the build specification. It supports four synthetic crops—caixin, pak choi, kailan, and lettuce—and forces recipe and row origin to synthetic. The broader catalogue contains twelve knowledge profiles, but the other eight crops do not have numerical recipes. This is a useful safety property: a literature record cannot silently become a production coefficient.

Private synthetic data lineage

The reference generator is `synthetic-farm-v1` in `fixtures.py`. It uses a fixed default cutoff (2026-09-08T00:00:00Z) and seed (20260908). It creates four demo recipes, sixteen 20 m² sheltered-hydroponic beds, eight already-executed batches, eight future orders per crop, twelve historical weeks per crop, one opening inventory lot, and declared farm resources. Every record that can carry an origin is labelled synthetic.

The Data Explorer may vary five bounded generator settings: historical multiplier, historical trend, repeating-pattern amplitude, order multiplier, and price multiplier. Its demand baseline uses `recipe-ewma-v1`; alpha is bounded from 0.05 to 0.95 and defaults to 0.35. For each crop, the forecast starts from the first point-in-time-eligible historical value, folds later values through an exponentially weighted moving average, retains booked orders by due date, and adds only the positive residual on the week end. This is an engineering baseline, not a learned production model.



Availability rules are applied before demand calculation. Historical values require `available_at <= cutoff` and a week before the planning date. Orders require `booked_at <= cutoff`; the private contract rejects an order booked after the farm cutoff. Dates are converted to Singapore civil dates for the planning horizon. The feature builder records dependency IDs for each forecast row and produces a Parquet artifact plus a manifest. The committed feature manifest records 32 synthetic features and the exact input, artifact, and private-snapshot hashes.

The planner does not mutate executed batches. It creates whole-bed candidates by recipe and date, reserves nursery sites from sowing to transplant, reserves a bed from transplant through harvest and sanitation, and budgets weekly labour and cash. A daily FIFO lot ledger keeps opening stock, harvest, delivery, disposal, and closing stock in mass balance; later harvest cannot fill an earlier delivery. The implementation and its exact policy weights are in `packages/planner/engine.py`. A companion numerical chapter should be used for formula-level interpretation.

Snapshot and scenario lifecycle

1. `/api/v1/bootstrap` creates an anonymous tenant and its first synthetic farm version when no valid session exists.
2. `/api/v1/imports` validates either the named reference fixture or a strict Farm payload, then appends a farm version. It never overwrites the prior version.
3. `/api/v1/planning-runs` freezes the latest farm payload, input hash, farm version, public source-card snapshot, evidence-register hash, market scaffold, and News context. An idempotency key prevents changed inputs from reusing the request identity.
4. The worker calculates all three policies against one forecast and scenario set. Persisted, sequenced events support server-sent-event reconnect.
5. A versioned backend policy can mark a feasible result `ACCEPTED_FOR_SIMULATION`. The result includes the synthetic ledger and simulated work; it does not authorize a farm operation.
6. Replay returns persisted results and events with `execution_mode=replay`. It does not calculate again or create a provider request.
7. A replan appends a new synthetic farm version and child run. If the farm changes during a run, acceptance becomes stale and a numerical-only refresh starts against the current version.
8. Scenario branches copy a farm or saved-playground root, apply bounded controls, and persist baseline and result. Comparisons reject branches with different roots, baseline hashes, forecast settings, or calculation versions.

Explorer snapshots add another integrity layer. A saved playground freezes the actual generated dataset, forecast, settings, generator/forecast versions, cutoff, reference snapshot, and hashes. Reload validates stored JSON against the recorded hashes. Unsupported old numerical versions fail explicitly rather than being recomputed under current code. Relevant implementation is in `data_explorer.py`; verification is recorded in `reports/data_explorer.md`.

Real public-data ingestion

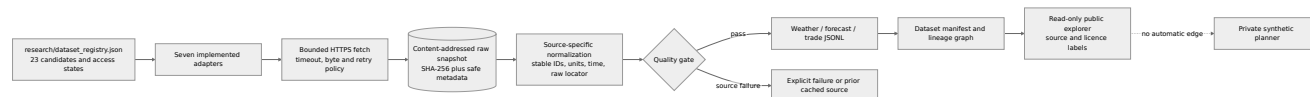
The source registry identifies 23 candidate datasets, but only seven connectors produced normalized observations in the recorded dataset build. “Listed” and “integrated” therefore mean different things.

Source	Implemented record type	Recorded release snapshot	Point-in-time/planning status
D01 NEA rainfall	Station rainfall, provider interval retained	88 rows	Observation available_at is uncertain; context only

Source	Implemented record type	Recorded release snapshot	Point-in-time/planning status
D02 NEA air temperature	Station air temperature	18 rows	Context only; not root-zone or on-farm temperature
D03 NEA relative humidity	Station relative humidity	18 rows	Context only; station microclimate differs from farm conditions
D04 NEA 24-hour forecast	Issued and valid forecast fields	20 rows	Eligible metadata exists, but no planner feature consumes it
D05 NEA four-day outlook	Four daily forecast groups	20 rows	Valid horizon is preserved and cannot be stretched over a crop cycle
D06 SingStat T010002	Monthly HS8 merchandise trade volume	15 rows	Exact release time unknown; broad codes are not crop demand; export blocked while reuse terms remain unverified
D11 NASA POWER	Seven days of gridded historical temperature, humidity, and precipitation	21 rows	Release timing unknown; historical grid context, not a farm sensor or forecast

The counts above come from the retained dataset manifest: 145 weather observations, 40 weather-forecast fields, 15 trade observations, 200 normalized rows total, and eight raw snapshots because D06 required an initial period probe and a larger response. The manifest was built on 8 September 2026 and is historical evidence, not a claim that the same observations are current now.

The implementation stores raw responses by SHA-256 under a source-specific directory with mode 0600, then records request URL, sorted query, retrieval time, byte count, media type, safe response headers, licence state, and relative path. It normalizes rows with stable IDs and raw locators. Quality validation rejects duplicate IDs, wrong units, non-finite values, naive timestamps, uncertain availability marked as point-in-time eligible, untraceable rows, and orphaned snapshot references.



Source-specific code is split among NEA normalization, SingStat normalization, NASA POWER normalization, snapshot storage, context orchestration, and quality validation. The machine-readable dependency graph is retained in data/manifests/lineage.json.

The public explorer only exposes curated fields. Callers cannot provide file paths. Lists, pages, queries, and exports are bounded; CSV cells beginning with spreadsheet formula characters are escaped. A source-level reuse policy enables export only when licence state is verified. Weather rows from D01–D05 are exportable under the recorded Singapore Open Data licence status;

D06 and D11 rows are view-only in the current configuration. Registry-only candidates appear as metadata without invented observations.

Rebuild semantics and an important reproducibility limit

`python scripts/build_dataset.py --data-dir data` performs a current live refresh. `--offline` rebuilds from raw snapshot files named in the manifest. Raw, normalized, and runtime data directories are intentionally excluded by `.gitignore`, while manifests and quality reports are committed. As a result:

- a live rebuild is reproducible at the code and transformation level, but it will fetch the provider's current responses and need not reproduce the 8 September row values or hashes;
- an offline byte-for-byte rebuild requires an authorized copy of the original raw snapshot directory, which a clean Git checkout does not contain;
- the committed manifest permits audit of hashes, row counts, source queries, transforms, and provenance, but does not independently permit recomputing every retained public value;
- public data licences and terms still govern redistribution even when a checksum is public.

This distinction should accompany any “reproducible dataset” claim.

News and current-information lifecycle

News is a deterministic supporting collector, not an eighth council agent and not a numerical feature pipeline. `packages/news.py` contains seven declared sources. Four are allowlisted RSS metadata collectors: SFA Newsroom, SFA Circulars, AFSIS agricultural-disaster reports, and ASEAN Agri-food. Reddit, SAFE, and the Central Singapore CDC calendar remain visibly unavailable, link-only, or blocked. The v5 release evidence records 346 valid metadata records (154 SFA Newsroom, 123 SFA Circulars, 64 AFSIS, and 5 ASEAN Agri-food); see the News implementation report.

The collector runs separately from the application and provider inference. It follows no redirects, refuses compressed responses, limits each response to 1 MiB and 200 items, validates a fixed HTTPS host, accepts XML media types, strips HTML to bounded text, and never fetches linked articles, images, or social profiles. It retains old records on a refresh failure and exposes the source failure state. A six-hour minimum attempt interval prevents repeated collection.

Each record preserves the exact raw publication string, parsed publication time, retrieval time, canonical URL, publisher geography, keyword-derived crop/topic tags, optional explicit structured event dates, reuse mode, and content hash. A date mentioned in a title is never inferred as an event date. Headlines are treated as untrusted context.

For a new decision, the API chooses a cutoff and copies a bounded News pack into the run or branch payload:

- historical replay uses the farm cutoff;
- a current synthetic decision uses its creation time while retaining the separate farm cutoff;

- both publication and retrieval must be at or before the chosen cutoff;
- continuation and child branches inherit the parent's frozen News context;
- replay reads the copied context;
- the pack declares `numerical_effect: none`.

There is no code path from a News record, headline keyword, social observation, public weather row, or trade row into EWMA demand, recipe yield, order quantities, or CP-SAT constraints. An advisor can cite a recognized frozen key, but any proposed numerical change must become an explicit bounded scenario and pass local validation.

Crop knowledge and scientific evidence

The version-controlled crop catalogue contains twelve unranked Singapore-relevant leafy-herb concepts. The evidence register contains 24 publication records (P01–P24). The catalogue-to-publication relation is many-to-many and preserves taxonomy status, aliases, harvested part, system relevance, evidence IDs, scope boundaries, and parameter gaps.

Crop knowledge profile	Linked evidence	Numerical demo recipe?	Major unresolved boundary
Caixin	P01, P02	Yes, synthetic	Local commercial mature marketable yield and cultivar cycle
Pak choi / xiao bai cai	P03–P05	Yes, synthetic	Species/cultivar-to-SKU mapping and commercial packout
Kailan	P05–P07	Yes, synthetic	Authority-resolved name/cultivar and commercial yield
Bayam	P10	No	Portfolio species, full-size yield, and multi-cut response
Kangkong	P12	No	Singapore commercial yield, cultivar, and regrowth
Lettuce	P08, P09, P16, P17, P19	Yes, synthetic	Cultivar/SKU, Singapore cycle yield, energy, and shelf life
Kale	P15	No	Taxon and cultivar remain blocked pending methods confirmation
Mustard greens	P11	No	Evidence taxon does not resolve all portfolio SKUs

Crop knowledge profile	Linked evidence	Numerical demo recipe?	Major unresolved boundary
Malabar spinach	P14	No	Singapore evidence, trellis capacity, and cut-specific yield
Sweet-potato leaves	P13, P18	No	Fresh-market leaf yield and local regrowth schedule
Garlic chives	P21, P22	No	Local accession, commercial yield, and cut-specific regrowth
Sawtooth coriander	P23, P24	No	Local seed source, flowering/regrowth, and buyer quality

The two v6 additions and their primary-study interpretation are documented in docs/research/v6_crop_evidence.md. The catalogue also links public government identifiers and names through source_registry.json, while crop_hs_mappings.json distinguishes exact product references from unresolved broad customs codes.

The evidence register is a targeted research set, not a systematic review or a training corpus. It records access-review status, study setting, finding, design implication, transfer limit, licence state, and a parameter_use decision. Most entries are restricted to context, candidate evaluation, or citation discovery. None of the 24 papers supplies the four numerical demo recipes. In particular:

- a non-Singapore controlled experiment does not validate Singapore commercial performance;
- biomass, dry weight, physiology, metabolomics, and nutritional quality are not interchangeable with fresh marketable kilograms;
- study measurements available only at harvest would leak future information into an earlier yield prediction;
- one review and its cited trial are not independent replications;
- a title, abstract, or inaccessible full text cannot authorize a numeric coefficient;
- broad HS trade categories cannot become named-crop customer demand;
- an illustration is representative artwork and does not resolve a cultivar, SKU, disease, or growth measurement.

The Crop Atlas returns the exact linked evidence records for a selected profile and displays the record's source URL and transfer limit. It also shows a planning recipe only for the four simulated crops and labels it demo_only. This makes the scientific layer inspectable without presenting it as numerical authority.

Persistence, tenancy, and lifecycle controls

The application uses one SQLAlchemy metadata collection and creates additive tables at service startup. The important persisted grains are:

Tables	Grain and lifecycle role
tenants, farm_versions	Anonymous session identity and append-only tenant farm versions
planning_runs, run_events	One idempotent planning request and its ordered event stream
scenario_branches, quest_progress	Isolated numerical experiment and per-tenant game progress
explorer_snapshots	Integrity-checked synthetic playground snapshot
conversations, conversation_messages, conversation_requests, conversation_events	Frozen advisor context, messages, idempotent provider request, and replayable events
council_research_sessions, council_research_jobs, council_research_actions	V7 isolated research state, one result per input version, and idempotent actions
inference_budget, rate_counters, security_settings	Durable global provider budget and abuse-control state

All user-state queries include `tenant_id`. Planning events and conversation children additionally use composite foreign keys that bind child identity to the same tenant as the parent. Unknown or unowned IDs return not found. Idempotency keys are scoped to the tenant or parent resource and reject reuse with changed request hashes. Tenant-row locking serializes farm-version allocation and competing creates.

Worker recovery reflects side-effect risk. Numerical scenarios return from `RUNNING` to `QUEUED` because recalculation is local and deterministic. An abandoned inference-bearing planning or conversation request is marked failed or interrupted so a restart does not silently repeat a paid call. Reconnect and replay consume stored events.

The v7 research route is a separately persisted study. Free text is parsed only into a small allowlisted set of proposals. An explicit apply action creates a new research input version, and calculation uses a deep copy of the synthetic farm. The main farm and its missions are not modified. The optional actual-advisor action is separately labelled and persists evidence-validation status. Implementation: `council_research.py` and `planner/research.py`.

API-to-interface traceability

The API is the only bridge from persisted or curated data into the React client. The public OpenAPI/Swagger endpoints are disabled, so source contracts and generated TypeScript types are the reviewable interface definition.

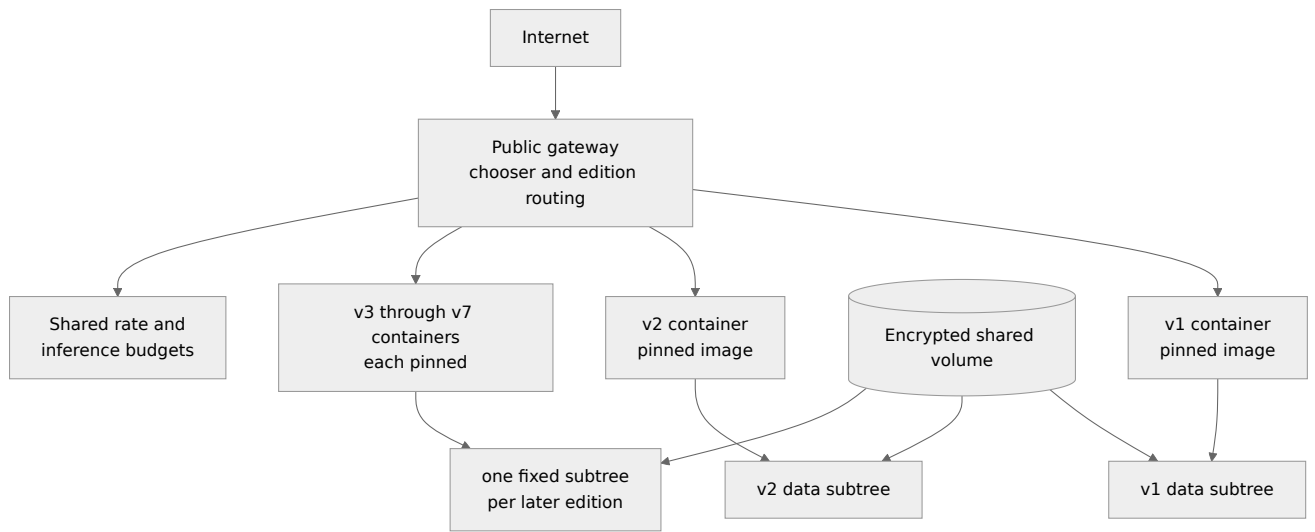
User surface	API source	What the UI can truthfully establish
Farm board and planning room	/bootstrap, /planning-runs, /events, /replay, /worklist.csv	Synthetic current farm, frozen inputs, locally calculated alternatives, event status, and simulation-only work
Crop Atlas	/crops, /crops/{id}/evidence	Catalogue identity, unresolved boundaries, demo recipe status, and linked evidence records
Data Explorer	/data-explorer/*	Selected tenant snapshot, records, charts, EWMA preview, exact hashes, same-root comparisons, and bounded exports
Public Explorer	/data-explorer/public*, /sources	Cached public rows, source time, retrieval time, freshness, licence state, and metadata-only candidates
News panel	/news	Frozen or current cache metadata with explicit time/relevance limits and no numerical effect
Scenario Lab	/scenarios, /scenarios/compare, /quests	Explicit synthetic control, immutable baseline, local result, and affected-resource pointers
Advisor conversations	/conversations/*	Frozen snapshot/evidence context, provider-request status, validated references, and stored replay
V7 council research	/council-research/*	Versioned scripted interaction, explicit proposal/apply lifecycle, local plans, challenges, and optional provider interpretation

The UI renders provider prose as text, opens external evidence links with safe browser attributes, and exposes structured provenance rather than provider internals. Static routes are fixed build directories; there is no route that accepts a filesystem path. Request bodies use strict Pydantic models with unknown fields rejected, and writes are same-origin checked and capped at 1 MiB.

Edition and deployment isolation

The current release registry marks v7 as latest and records a full source commit and immutable OCI image digest for every v1-v7 edition. Since v6, one Singapore Fly Machine and one encrypted volume host the gateway and all edition containers. Sharing a host is not sharing application state: the host adapter binds a fixed edition subtree to /data, removes access to the common parent, and each container maintains its own PostgreSQL cluster, cache, session state,

and game records. The gateway owns shared abuse counters and the 48-call daily inference ceiling because those are operational safeguards rather than game progress.



Cross-edition cookies are rejected and published source/image identities are immutable. A new edition begins with fresh reference state. The single Machine and volume remain a shared availability and capacity boundary: a host restart can interrupt every edition, and the v7 verification is a bounded two-job smoke test rather than a saturation or long-duration study. Details and recovery procedure are in docs/deployment/editions.md; v7 preservation and capacity evidence are in reports/v7/implementation.md.

Evidence strength and claim limits

Claim	Best supporting artifact	What the artifact does not establish
The public pipeline produced 200 traceable rows from seven source IDs	dataset manifest, lineage graph, quality report	Current freshness, farm relevance, commercial prediction accuracy, or redistribution permission for every source
The feature builder excludes public context from the demand baseline	feature manifest, build_features.py	That public signals would be useless after future validation; only that they are unused now
Twelve crop profiles link to 24 publications	crop catalogue, evidence register	Exhaustive literature coverage, independent replication, local transferability, or approved production coefficients
The News collector retained 346 valid metadata records for v5	News report	Full-article verification, sentiment, causal impact, representative public opinion, or current availability
Explorer snapshots and scenarios persist and replay	Data Explorer report, v7 persistence evidence	Durability under arbitrary infrastructure failure or migration from unsupported future model versions

Claim	Best supporting artifact	What the artifact does not establish
Prior edition records survived v7 publication	v7 release report	Multi-region availability or unlimited shared-host capacity
An actual v7 advisor request completed	v7 advisor probe	Correct farming advice; the recorded response had no verified evidence references and was marked unsupported

Automated checks provide strong evidence for schema enforcement, deterministic calculations, idempotency, tenant isolation, browser flows, hashes, and bounded release preservation. They are weaker evidence for scientific validity, human usability, and real operations. Agent walkthroughs are not human studies. Headless mobile geometry is not physical touch or software-keyboard testing. Digital audio checks are not proof of audibility on a farmer's device. Synthetic held-out weeks generated by the same fixture process do not measure out-of-sample commercial accuracy.

No real farm outcome dataset is present. There is no trained supervised demand model, harvest/yield model, disease classifier, satellite predictor, calibrated probabilistic risk model, production retraining pipeline, drift monitor, causal estimator, or measured SME benefit. EWMA is a statistical baseline, CP-SAT is an optimizer, and scenario weights are declared assumptions. "AI" in this release refers to optional qualitative advisor calls; it does not replace data collection, model validation, or numerical computation.

Reproduction and audit route

From a dependency-complete checkout, the main read-only or local-data reproduction commands are:

```

# Validate the committed synthetic fixture and rebuild its point-in-time features.
.venv/bin/python scripts/build_features.py

# Fetch current bounded public observations and write a new manifest.
.venv/bin/python scripts/build_dataset.py --data-dir data --with-power

# Rebuild from an authorized local copy of the raw snapshots named in the manifest.
.venv/bin/python scripts/build_dataset.py --data-dir data --offline

# Refresh the bounded RSS metadata cache; no inference is involved.
.venv/bin/python scripts/refresh_news.py

# Run focused data, private-contract, explorer, and isolation verification.
.venv/bin/python -m pytest -q \
  tests/contracts/test_private_data.py \
  tests/contracts/test_features.py \
  tests/data/test_ingestion.py \
  tests/data/test_explorer_public.py \
  tests/explorer/test_data_explorer.py \
  tests/review/test_postgres_concurrency.py

# Verify the generated browser contracts and production client build.
.venv/bin/python scripts/generate_web_contracts.py --check
npm run build --prefix apps/web

```

The public-data commands access changing upstream services, so record retrieval time, response status, row counts, failures, and new hashes rather than expecting the historical manifest's values. PostgreSQL concurrency tests require a dedicated test database and must never target a live application database. Running the commands above does not require or authorize a provider call.

Material gaps found in this audit

The following discrepancies were found against the audit baseline. This documentation update corrects stale prose and specifies code fixes; it does not claim the listed code gaps are repaired:

1. `packages/ingestion/validation.py` still hard-codes `catalogue_profiles: 10` and `scientific_evidence_documents: 20`. The current registries contain 12 profiles and 24 documents. Consequently, the committed data quality report is historically accurate for the 8 September build but stale as a current coverage statement. Coverage should be derived from the registries or clearly versioned as build-time metadata.
2. `FarmTact_Build_Specification.md` labels its v6 section “Current implementation,” says the shared host runs v1-v6, and says the evidence register has 20 publications. V7 is the current published edition and the register now has 24 publications. Historical descriptions should be labelled historical and the current amendment consolidated at the top.
3. `docs/execution-plan.md` correctly names v7 as current, but its unqualified “Acceptance scope” still says ten crop profiles and its early build summary says ten profiles/twenty

records/six roles. Those statements describe the original milestone and need an explicit “historical v1 scope” qualifier where they appear.

4. docs/deployment/fly.md describes v6 and v1–v6 as the current topology. It should defer to the v7 editions runbook or identify itself as a v6 historical snapshot.
5. docs/v5-engagement-plan.md says v6 is current. Its historical-plan label is useful, but the “current” sentence should point readers to v7.
6. docs/research/v5_news_sources.md refers to coverage broader than “the ten crops.” The source was a v5 research brief, but the current shared crop vocabulary has twelve identifiers; either freeze the statement as v5-era scope or update it.
7. The build specification's recommended architecture mentions Next.js, DuckDB, an object-store abstraction, and broader operational tables. The implementation uses Vite React, FastAPI, PostgreSQL JSON payloads, local content-addressed files, and a much narrower synthetic Farm contract. This is a roadmap-versus-implementation difference and should be stated where the architecture recommendation appears.
8. The committed public manifest points to ignored raw and normalized files. The documented offline rebuild is therefore not self-contained in a clean checkout. The docs should say that an authorized raw-snapshot bundle is required for exact historical reproduction.

These gaps do not invalidate the implemented synthetic demonstration. They narrow what can be claimed from its current artifacts and make clear which records are historical evidence, which are active contracts, and which remain future design requirements.

The specification/documentation wording findings above refer to the audit baseline. The integrated documentation update corrects current pointers and labels historical requirements; numerical, data-generation and runtime gaps remain future work.

Implementation gaps and next-iteration acceptance criteria

Audit: 11 September 2026, source baseline a96025e; deployed v7 remains frozen. This register records observed implementation gaps and scientific limitations. The proposed changes are **requirements for future work**, not fixes implemented by this documentation update. P0 means address before making the affected correctness claim or using the feature as reliable evidence; it does not mean real farm operations are enabled. P1 improves the next demonstration; P2 requires a separately validated data/research expansion.

Source detail: Council/provider audit, numerical audit, game backend audit, and data architecture audit.

1. Council, evidence and provider behavior

ID / priority	Observed gap and source	Required next behavior / acceptance evidence
AI-01 / P0	Both v7 actual adviser probes remain unsupported; transport/replay success is not answer-quality success. Local, public.	Build frozen, expert-reviewable cases with expected references and permissible conclusions. Evaluate role relevance, unsupported quantities, contradictions, abstention and useful answers separately from HTTP/schema success. Preserve all failed attempts and per-case usage. Do not hide unsupported replies or use them to accept a plan.
AI-02 / P0	Reference membership and numeric-value matching do not prove that cited evidence entails a sentence. Claims, conversation validation.	Represent quantitative assertions as typed reference/value/unit/entity/period relations and render those values from code. Test wrong crop/date/unit/sign, a correct number attached to the wrong meaning, fabricated causal claims and citation laundering. Qualitative statements retain an explicit unverified status unless independently supported.

ID / priority	Observed gap and source	Required next behavior / acceptance evidence
AI-03 / P1	Three mechanisms share “council” language: precomputed planning findings, bounded conversation turns and scripted research dialogue.	Expose workflow type, inference origin, model/call status and whether a new calculation occurred. Tests must distinguish a script, fixture, actual completion, stored actual replay, failed provider call and numerical-only result. No invented tool or agent activity.
AI-04 / P0	Capability verified is derived from archived trial JSON plus key presence for text; vision uses archived success. views.capabilities.	Return historical probe date/model/source, configured/not-configured state and last observed execution separately. Missing/aged/account-mismatched evidence must not claim current verified access. Test old reports, no key, missing model and failed last execution without triggering paid background probes.
AI-05 / P1	Registered helper, evaluator and vision role names exceed actual integrated call sites. Runtime manifest.	Maintain a machine-readable route-to-caller/feature inventory. Distinguish available server capability from active product integration. A new helper/evaluator requires caller, prompt/schema version, finite budget, test, egress review and visible failure semantics.
AI-06 / P1	Per-run request and reserved-output limits plus shared call counts do not establish a hard USD spending cap or strict end-to-end cancellation. Gateway.	Add conservative total-cost reservation with dated price metadata if a monetary guarantee is required; report input/output/cache/reasoning accounting. Test timeout/cancellation during transport and between turns, midnight reconciliation and multi-edition concurrency. Do not automatically retry potentially billed interrupted generations.
AI-07 / P1	Crop-harvest forecast emits marketable_kg; the conversation reference builder expects expected_kg, omitting that forecast mass reference. Forecast, context builder.	Adopt one shared typed harvest record and reference generator. Assert that every forecast batch/date/marketable-mass field is available with the same value, unit and frozen snapshot hash in adviser context. Preserve existing batch-level references during migration.

| AI-08 / P1 | Mission specialists see the same frozen tools but not one another's findings; only the chair sees previous validated/rejected findings. No two-round challenge protocol exists. | Preserve this truthful sequential contract or implement explicit challenge targets, turn/round budgets, stop states and evidence updates. Test which prior findings and rejection reasons each

role receives; measure usefulness against the simpler existing chain. | | AI-09 / P1 | Conversation prompts request tighter response limits than AdvisorReply enforces (400 versus 900 characters and lower reference/action counts). | Put one bounded response contract in the schema and generate prompt instructions from it. Test limits at and just beyond every boundary; distinguish structural repair from unsupported-claim handling. | | AI-10 / P1 | Conversational chair sees prior validation_status but its context projection omits detailed validation_errors; mission chair receives rejection reasons. | Define a bounded shared validated-message projection with explicit error categories and test that later roles cannot treat rejected/unsupported statements as evidence. | | AI-11 / P1 | Per-call records lack a complete explicit prompt/schema/validator version set; roster version alone cannot identify prompt behavior. | Persist prompt-template, output-schema, validator, numerical/context and source versions with every completion and replay export. Demonstrate reconstruction of the sent public context without persisting private reasoning. | | AI-12 / P1 | A persistent conversation request may be COMPLETED while every reply is unsupported; a valid mission claim can withhold selection through its recommendation field. | Expose execution status, evidence status and decision influence separately. Test all-successful transport with unsupported answers, partial failures and a single veto; compare with a numerical-only run on identical inputs. |

Council-specific chain, validation and budget gaps are analyzed in the AI chapter. The game's challenge/proposal/acceptance state gaps are separately identified as GAME-* in the backend chapter; those are part of this register's next-iteration scope and must not be lost in a provider-only assessment.

2. Crop development, optimization and simulation

ID / priority	Observed gap and source	Required next behavior / acceptance evidence
NUM-01 / P0	Backend marks batches ready at/after harvest; standalone frontend preview makes beds empty at/after harvest. Neither exposes sanitation occupancy. views, World.	Define one versioned crop-cycle state contract for nursery, growing, ready, harvested, sanitation and available, distinguishing planned versus recorded events. Test every day around transplant, harvest and sanitation end, with/without accepted overlays and delayed batches. UI availability must agree with planner occupancy.
NUM-02 / P1	Preview slider includes planning_date + horizon_days, one day beyond the final planner index.	Use the same inclusive horizon contract everywhere. Test a 7-, 56- and 84-day horizon, date labels, exports and final-day inventory.

ID / priority	Observed gap and source	Required next behavior / acceptance evidence
NUM-03 / P1	Three scenario weights are stored but not consumed by the solver; seed produces no random draws. models, engine.	Either enforce equal-weight-only configuration explicitly or consume validated normalized weights. Demonstrate changed weights affect a constructed optimum; report scenario provenance. Rename/remove unused random-seed semantics until stochastic sampling exists.
NUM-04 / P1	“Resilient” increases shortage penalties; it is not maximin, CVaR or a low-yield service guarantee. The objective is policy-weighted utility, not literal expected net profit.	Publish objective terms and units. If downside protection is the desired product contract, implement an explicit risk/service constraint with infeasibility reporting. Test trade-offs on small enumerated cases and compare all policies on the same baseline.
NUM-05 / P0	CP-SAT inventory age buckets do not explicitly require FIFO; the reporting simulator uses FIFO. This is an unproven equivalence, not a demonstrated failing numerical case.	Prove equivalence for the supported domain or align both allocation rules. Enumerate small perishability cases and reconcile delivered mass, expiry, revenue and selected objective against replay. Keep this classified as a validation gap until a counterexample or proof exists.
NUM-06 / P1	General terminal stock value/penalty is absent; horizon truncation can affect the preference for unconsumed stock.	Declare terminal treatment and test horizon extension, late harvest, salvage assumptions and expiry beyond the horizon. Never classify all closing stock as physical waste.
NUM-07 / P1	Residual demand with no booked line in its week is assigned zero price. Fulfilment is by crop/date, not individual buyer/order/grade.	Make missing prices explicit and distinguish requested, booked and delivered quantities. Add buyer-level allocation only with corresponding schema/priority rules; test scarce stock with multiple prices and cancellation states.
NUM-08 / P2	No fitted maturity/yield/growth/quality/weather-response model; delay/yield shocks are direct inputs without biological coupling.	Collect independent crop-cycle outcomes and availability timestamps. Compare recipe baselines with candidate models on whole-batch temporal holdouts; report timing/yield errors and interval coverage. No model promotion from synthetic accuracy alone.

ID / priority	Observed gap and source	Required next behavior / acceptance evidence
NUM-09 / P2	EWMA is constant-level and tested only on patterned fictional history; no calibrated demand distribution.	Rolling-origin multi-horizon real-order evaluation with last-week and seasonal baselines, leakage checks and stockout/cancellation handling. Separate farm/buyer cohorts and report uncertainty in validation estimates.
NUM-10 / P2	Whole-bed, single-harvest assumptions omit water, energy, HVAC, seed, packaging, grade, rotation, multi-cut, partner supply and buyer-level contracts.	Maintain a supported-constraint registry. Extend schema, validator, solver, replay and UI together for each capability. In particular, multi-cut crops require persistent occupied capacity and regrowth states, not another single-cut recipe.

3. Evidence, provenance and engineering reproduction

ID / priority	Observed gap and source	Required next behavior / acceptance evidence
DATA-01 / P0	Quality builder hard-codes 10 profiles/20 publications; current registries contain 12/24. validation.py.	Derive coverage counts from validated input registries, version manifests and fail on contradictory totals. Preserve old release reports as dated artifacts rather than rewriting them as current evidence.
DATA-02 / P1	integrated_demo.py still expects 10 crops, while the catalogue contains 12.	Derive expected coverage from a versioned manifest and distinguish recipe count from knowledge count. An integrated run must pass without weakening tests to accept arbitrary missing crops.
DATA-03 / P1	The checked-in numerical report has evaluation_scope metadata that its current generator does not emit.	Add a versioned report schema and generator check. Regeneration must preserve cohort, settings, fixture hash, cutoff, model versions and explicit synthetic-only conclusions. Archive before replacing evidence; report solver nondeterminism where applicable.
DATA-04 / P1	Ignored raw/public cache files prevent an exact historical rebuild from Git alone.	Produce a licensed, integrity-checked snapshot bundle or clearly offer transform-only reproduction. Compare source byte hashes and availability cutoffs; a new live fetch is a new dataset version.

ID / priority	Observed gap and source	Required next behavior / acceptance evidence
DATA-06 / P1	Fresh offline game checks fail because test_seven_current_advisors_and_frozen_create_idempotency assumes an ignored NEA D04 cache is present (source:D04.summary).	Give offline tests explicit bounded source fixtures or assert the documented missing-source state; keep real-source integration checks separate. A clean checkout must pass its declared offline group without fetching current data to satisfy a fixture assertion.
DATA-05 / P1	Citation/source presence does not validate numerical crop coefficients; public sources are contextual only.	Preserve public_features_used=[] until an explicit feature path and point-in-time validation exist. Any new weather/News/market numerical effect needs units, availability, exposure mapping, ablation and evidence of predictive benefit.
OPS-01 / P1	Python egress policy is not an OS/network enforcement boundary; a single host shares availability across editions.	Before operational promotion, validate infrastructure egress controls and recovery on an isolated environment. Exercise provider denial and database restore, not only static string scans or volume existence.
UX-01 / P1	AI/headless walkthroughs do not establish human usability or agronomic decision benefit.	Pre-register a small task-based human study with novice and relevant practitioner groups, clear synthetic framing, task success, time, error and comprehension outcomes. Do not infer benefits from animated dialogue or agent agreement.

4. Proposed bounded implementation sequence

1. **Restore semantic consistency:** AI-04, AI-07, NUM-01/02, DATA-01/02/03 and the concrete game-state inconsistencies in GAME-*. These improve truthful status, synchronized state and reproducible evidence without pretending to improve biology.
2. **Make the council useful and inspectable:** AI-01/02/03 and the Council backend gaps. Test interpretations against fixed quantities and explicit unresolved assumptions; make proposal → apply → calculation → compare → selection observable.
3. **Strengthen numerical acceptance:** NUM-03/04/05/06/07 with enumerated small examples, explicit policy semantics and objective/replay reconciliation.
4. **Evaluate people and data:** UX-01 and NUM-08/09/10 only with appropriate observed data and research design. Keep operational integrations disabled.

This is a recommended scope order, not authorization to publish a release as part of the documentation task. An implementation release must use the next unused immutable edition

and retain previous editions' source/image and state.

Reproducing the implementation report

The report audits source baseline a96025e; released v7 uses the source/image in config/releases/v7.json. Documentation and code can evolve after an edition is frozen. Record which source is being checked.

1. Distinguish the kinds of reproduction

Activity	Dependencies	Changes / external calls
Read source and archived JSON	Git checkout	None
Regenerate report figures	Python application requirements; optional CairoSVG + native Cairo for PNG	Writes documentation figures/data; no database, solver or network call
Offline provider/contracts tests	Locked Python requirements	In-process HTTP mocks; no real inference
Local numerical checks	Locked Python requirements	Real local solver; no inference; timings/feasible schedules can vary
PostgreSQL lifecycle/concurrency tests	Explicit dedicated local test databases	Writes isolated test state; never target a live farm database
Browser journeys	Built frontend, running dev service, Playwright Chromium	Real API/numerical work; some scripts intercept adviser fixtures; check each script
Public-data rebuild	Approved public source access or archived raw bytes	Live fetch creates new snapshots; offline needs raw files absent from Git
Authenticated DeepSeek trial	Explicit server credential, registered model access, finite budget	Metered calls; not needed for documentation reproduction
Edition publication	GitHub/Fly access and clean committed source	Updates hosting and immutable release registry; not part of this audit

2. Local application prerequisites

Use Python 3.13+, Node 24, PostgreSQL 18, the committed requirements.lock.txt and web lockfile. From repository root:

```
python -m venv .venv
.venv/bin/pip install -r requirements.lock.txt
npm ci --prefix apps/web
npm run build --prefix apps/web
```

Provision a **development** database and supply `FARMTACT_DATABASE_URL` through the process environment. `Store()` defaults to the Sprite-specific Unix socket `/tmp/farmtact-pg` and database `farmtact`; it does not create the PostgreSQL server or database. `.env.example` lists settings; the entrypoint does not automatically read a `.env` file. The application otherwise defaults to test execution and synthetic data; a key must not switch it to real farm operations.

```
.venv/bin/python scripts/initialize_database.py
.venv/bin/python -m packages.fixtures
.venv/bin/python scripts/build_dataset.py --with-power
.venv/bin/python scripts/build_features.py
```

The fixture command writes the fixture and shared schema. Feature/dataset commands write artifacts; do not mistake regeneration for a read-only check. Raw and normalized public files are ignored. `build_dataset.py --offline` requires already retrieved bytes and cannot reconstruct historical source contents from a manifest alone.

Outside Sprite, run `.venv/bin/python scripts/serve.py`. Inside Sprite, register it through `sprite-env services create` using the platform skill; do not create an unmanaged background server. For example, after provisioning the database service:

```
sprite-env services create farmtact-doc-dev \
  --cmd /home/sprite/workspaces/Code2/.venv/bin/python \
  --args /home/sprite/workspaces/Code2/scripts/serve.py \
  --http-port 8080
```

Use absolute paths appropriate to the actual checkout. This example does not provision the database or credentials. Inspect service CLI help for dependency registration and logs. The public Sprite URL must serve only the application, never repository roots, dumps or private files. No local HTTP service was required or started during this documentation audit.

3. Offline and numerical verification

These representative groups avoid a running PostgreSQL server and metered calls. The separate broader conversation test module has a known implicit D04 public-cache dependency (DATA-06); its failed fresh-checkout assertion is preserved in the audit:

```

.venv/bin/python -m pytest -q \
  tests/deepseek/test_gateway.py \
  tests/contracts/test_private_data.py \
  tests/contracts/test_features.py \
  tests/models/test_explorer_numerics.py \
  tests/seven_agents/test_council.py \
  tests/review/test_provider_isolation.py

.venv/bin/python -m pytest -q \
  tests/numerical/test_planner.py \
  tests/planner/test_research_constraints.py

.venv/bin/python -m pytest -q \
  tests/council_research/test_sessions.py \
  tests/council_research/test_independent_review.py \
  tests/gameplay/test_scenarios.py

.venv/bin/python scripts/generate_web_contracts.py --check

```

The full `pytest -q` suite also includes PostgreSQL recovery/concurrency tests. Inspect each database fixture first. Research tests explicitly require a separate `farmtact_research_test` database; other historical test modules use their documented local test configuration and may recover queues. **Do not run them against a running application database.** SQLite tests establish local behavior but do not prove PostgreSQL lock/concurrency semantics.

`node tests/browser/run.mjs` is the original journey. Later features have separate scripts under `tests/browser/` and their release reports record invocation details. Install Chromium with `cd apps/web && npx playwright install chromium` when needed. An intercepted adviser fixture verifies UI behavior, not DeepSeek response quality.

`scripts/evaluate_numerical.py` rewrites `reports/numerical_evaluation.json`. Run it only when intentionally producing a new versioned evaluation artifact. Its four-second per-policy solver limit means exact allocations, bounds and timing are not guaranteed identical between environments. Preserve the archived report and compare constraints/semantics, not only JSON equality. The metadata gap DATA-03 must be fixed before treating regeneration as complete evaluation provenance.

4. Report figures and rendering

The committed SVG/PNG figures can be read without Python. To regenerate SVGs and source data from the fixture:

```

.venv/bin/python docs/technical/figures/generate.py

```

For PNGs, install the optional documentation tools in a separate environment using `requirements-docs.txt`, plus the native Cairo library and DejaVu fonts supplied by your operating system. These packages are not runtime application dependencies. Then run the generator with

--png. The figure generator imports the locked application's fixture/contracts/forecast modules and makes no provider, source-ingestion, database or optimization request.

The figure data records the fixture hash, crop parameters, caixin observations and one-step EWMA values. Architecture arrows are authored annotations from source inspection. SVG text includes descriptions for accessibility. The figures were visually inspected as PNGs during the documentation audit.

The committed diagram manifest records the 20 rendered Mermaid SVGs and hashes of their chapter source blocks. To regenerate those images, use a separate documentation tool directory:

```
npm install --prefix /tmp/farmtact-doc-tools \  
  @mermaid-js/mermaid-cli@11.17.0 mathjax-full@3.2.1  
node /tmp/farmtact-doc-tools/node_modules/puppeteer/install.mjs  
.venv/bin/python docs/technical/render_diagrams.py \  
  --mmdc /tmp/farmtact-doc-tools/node_modules/.bin/mmdc
```

Chromium requires its native system libraries. A restricted local VM may require an explicit Puppeteer launch configuration; --puppeteer-config passes that file to Mermaid CLI. This audit used a local --no-sandbox browser solely for trusted authored documentation, with no public HTTP service. Mermaid parsing caught one sequence-label semicolon error, which was corrected before all 20 diagrams rendered.

Render the report with Python Markdown and the optional local MathJax module to produce a single printable HTML document:

```
.venv/bin/python docs/technical/render_report.py \  
  --mathjax-module /tmp/farmtact-doc-tools/node_modules/mathjax-full \  
  --output /tmp/farmtact-technical-report.html  
python docs/technical/check_docs.py
```

The renderer embeds scientific figures, Mermaid SVGs, existing screenshot evidence and MathJax-rendered equations. It includes all seven chapters and makes no network request. Without --mathjax-module, TeX remains source notation; without rendered diagram files, Mermaid remains source notation. The committed reading copies use both renderers. Source/evidence links point to GitHub; there are no external scripts, fonts or CDN dependencies in the HTML. The PDF exporter can print that HTML using local Puppeteer and also checks image loading, internal links and page overflow. See the audit record for the actual invocation and limits.

5. Authenticated trials and archival discipline

The DeepSeek runbook describes the bounded real trial. --live means authenticated transport, not operational farm mode. Never print a credential or include it in command arguments. Provider failure must remain blocked/failed; do not substitute another provider.

Keep separate records for source inspection, offline mocks, fresh authenticated calls and archived replay. A full suite pass does not validate crop biology or human decision benefit. The exact checks run for this audit, including counts, limitations and generated files, are in the audit record.